

# Processi Comunicanti



# Processi Comunicanti



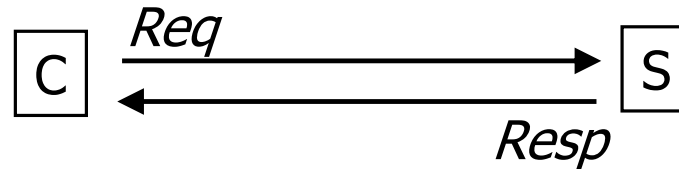
- ❑ **Processo** = Programma in esecuzione
- ❑ Processi possono:
  - ❑ Essere **distribuiti** su calcolatori diversi
  - ❑ **Comunicare** tra loro
- ❑ Calcolatori **localizzati** potenzialmente "**ovunque**"

# Client e Server (I)



- ❑ Processo **server**:  
Offre un "servizio" ad altri processi
- ❑ Processo **client**:  
Usa un "servizio" offerto da altri processi
- ❑ Ultra-ultra-semplificato:
  - ❑ Client trasmette **richiesta** a Server
  - ❑ Server
    1. Riceve richiesta
    2. Esegue "servizio"
    3. Trasmette **risposta** a Client

# Web (ultra-ultra-semplificato) (I)



Processo Client =  
Browser in esecuzione  
(*Chrome, Firefox, Safari, Edge,...*)

Processo Server =  
"Web Server"  
(*Apache, Tomcat, ...*)

## ❑ Servizio

- ❑ Prelievo documenti memorizzati sul web server
- ❑ Esecuzione programmi sul web server
- ❑ Interazione con database sul web server
- ❑ ...

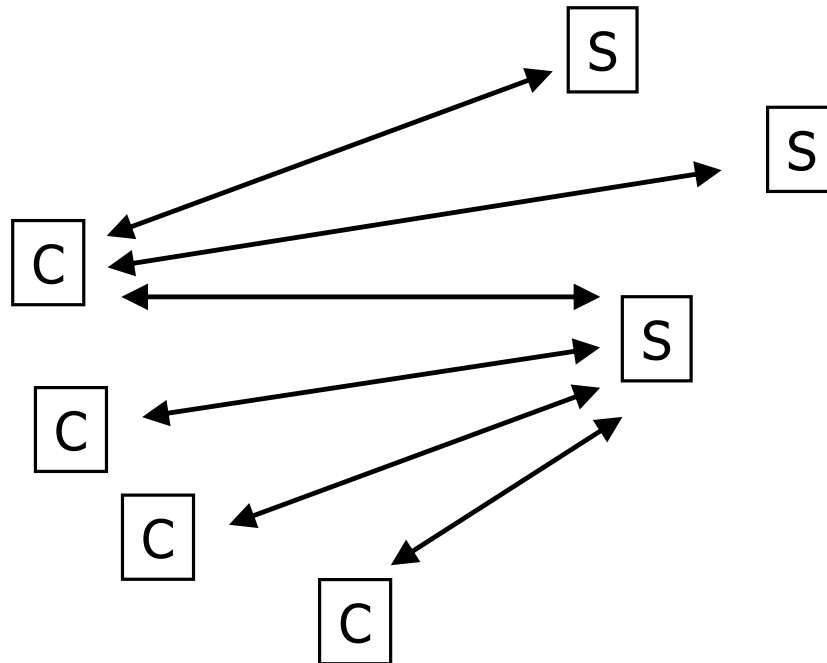
# Client e Server (II)



- ❑ Processo **server**:  
Offre un "servizio" ad altri processi
- ❑ Processo **client**:  
Usa un "servizio" offerto da altri processi
- ❑ In uno stesso "intervallo di tempo":
  - ❑ **Un client** può usare servizi di **molti server** diversi
  - ❑ **Un server** può offrire un servizio a **molti client** diversi

# Web (ultra-ultra-semplificato) (II)

- In uno stesso "intervallo di tempo":
  - **Un client** può usare servizi di **molti server** diversi
  - **Un server** può offrire un servizio a **molti client** diversi



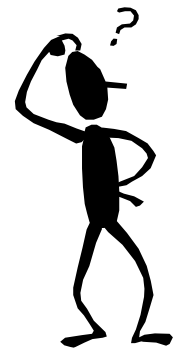
# Client e Server (III)

- ❑ In uno stesso "intervallo di tempo":
  - ❑ **Un client** può usare servizi di **molti server** diversi
  - ❑ **Un server** può offrire un servizio a **molti client** diversi
- ❑ **Un** processo può operare **sia da client sia da server**
  - ❑ In uno **stesso** "intervallo di tempo"
  - ❑ In "intervalli di tempo" **diversi**
- ❑ Vedremo esempi più avanti
  - ❑ Descrizione C/S di alcune app reali molto complicata (es: Whatsapp, Skype; non la vedremo)

# Hmmm...

- ❑ **Processo** = Programma in esecuzione
- ❑ Processo **server**:  
Offre un "servizio" ad altri processi
- ❑ "Hanno dei **server** potentissimi"
- ❑ "Si è guastato il **server**"
- ❑ "Ho comprato un **server**"

*Che vuol dire?*



# Terminologia



□ Nel gergo tecnico server può indicare

**1. Processo**

**2. Calcolatore** la cui funzione principale è  
l'esecuzione di processi server attivi 24x7

□ La differenza è evidente dal contesto

□ Per noi è (quasi sempre) un processo

# IMPORTANTISSIMO



❑ Entità di **tipo** radicalmente diverso

❑ Processo             $\neq$             Calcolatore

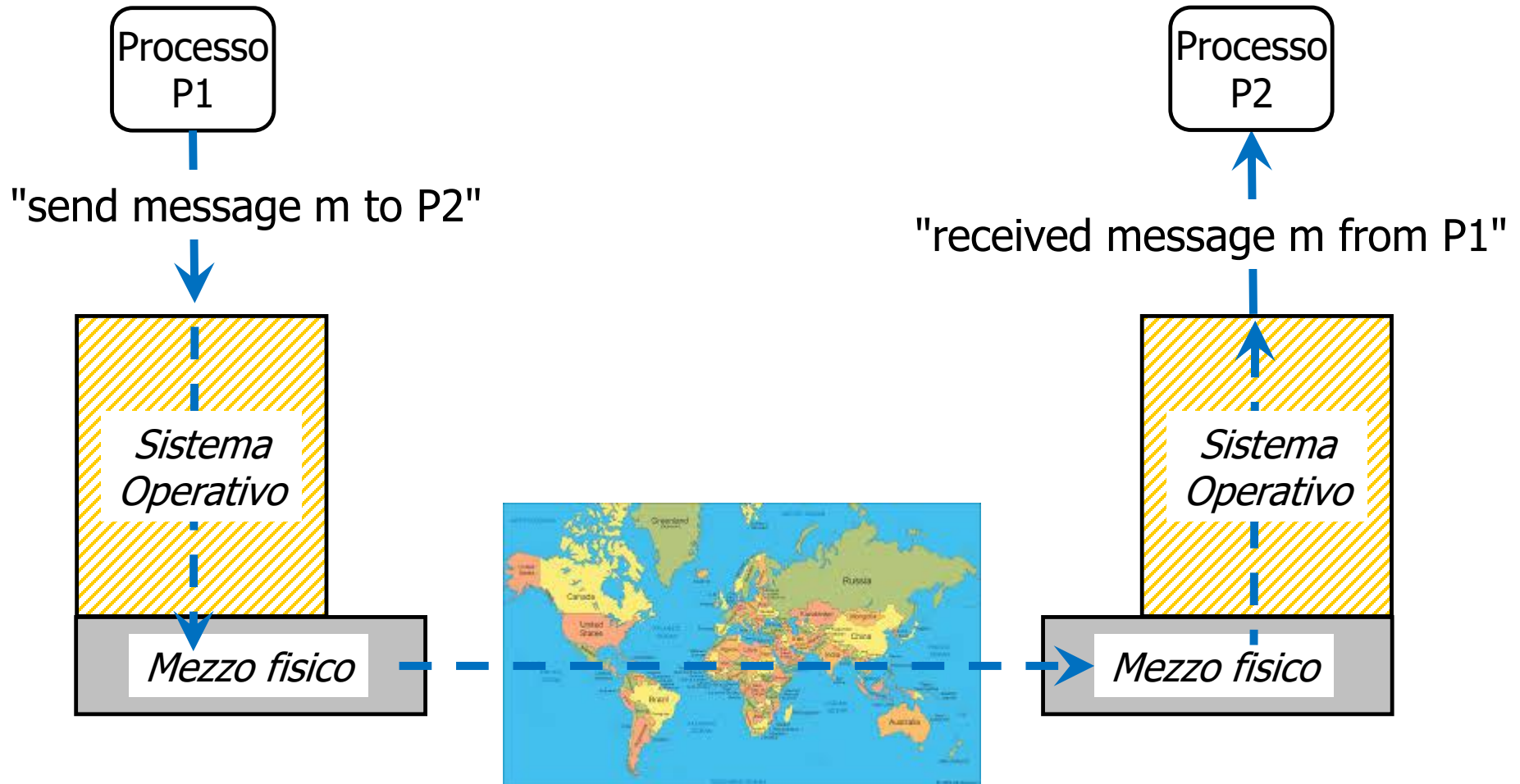
❑ Scarpa             $\neq$             Mela

❑ Sedia             $\neq$             Automobile

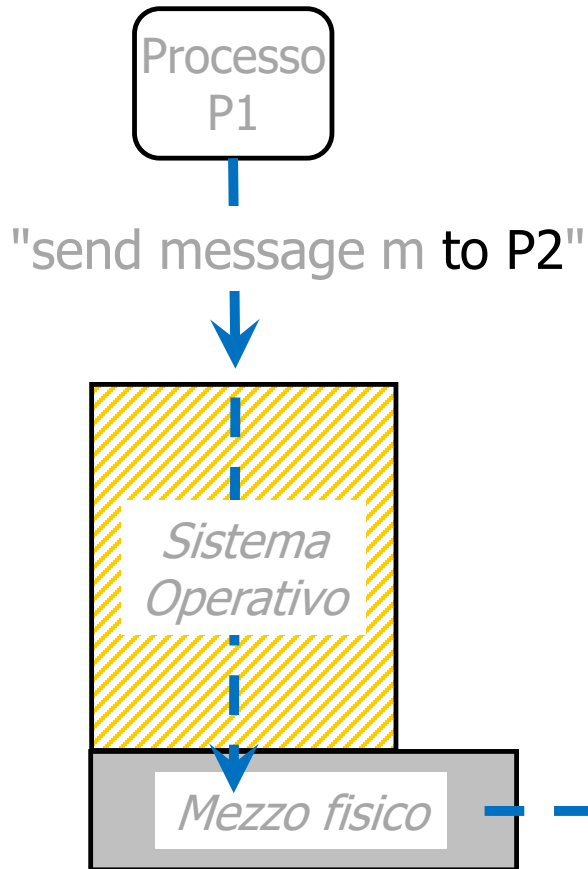
❑ La "scarsa attenzione" verso il **tipo** delle entità  
è una delle lacune principali degli studenti

❑ Importantissimo in questo corso

# Comunicazione



# Hmmm...



Come viene **localizzato**  
il calcolatore su cui  
è in esecuzione P2?



# Risposta

## APPARENTEMENTE banale

- ❑ Come si **localizza** il calcolatore su cui si trova il processo con un dato identificatore?
  - ❑ Come trovare il calcolatore su cui è in esecuzione P2?
- ❑ **Non ci interessa**
- ❑ **FONDAMENTALE** imparare a pensare in maniera **modulare**
  - ❑ Come si **usa** (interfaccia)
  - ❑ Come **funziona** (implementazione)
- ❑ Quando accendo l'auto mi interessa sapere come funziona?
- ❑ Quando regolo la temperatura del frigorifero?

# Internet (Ultra-ultra-semplificato)



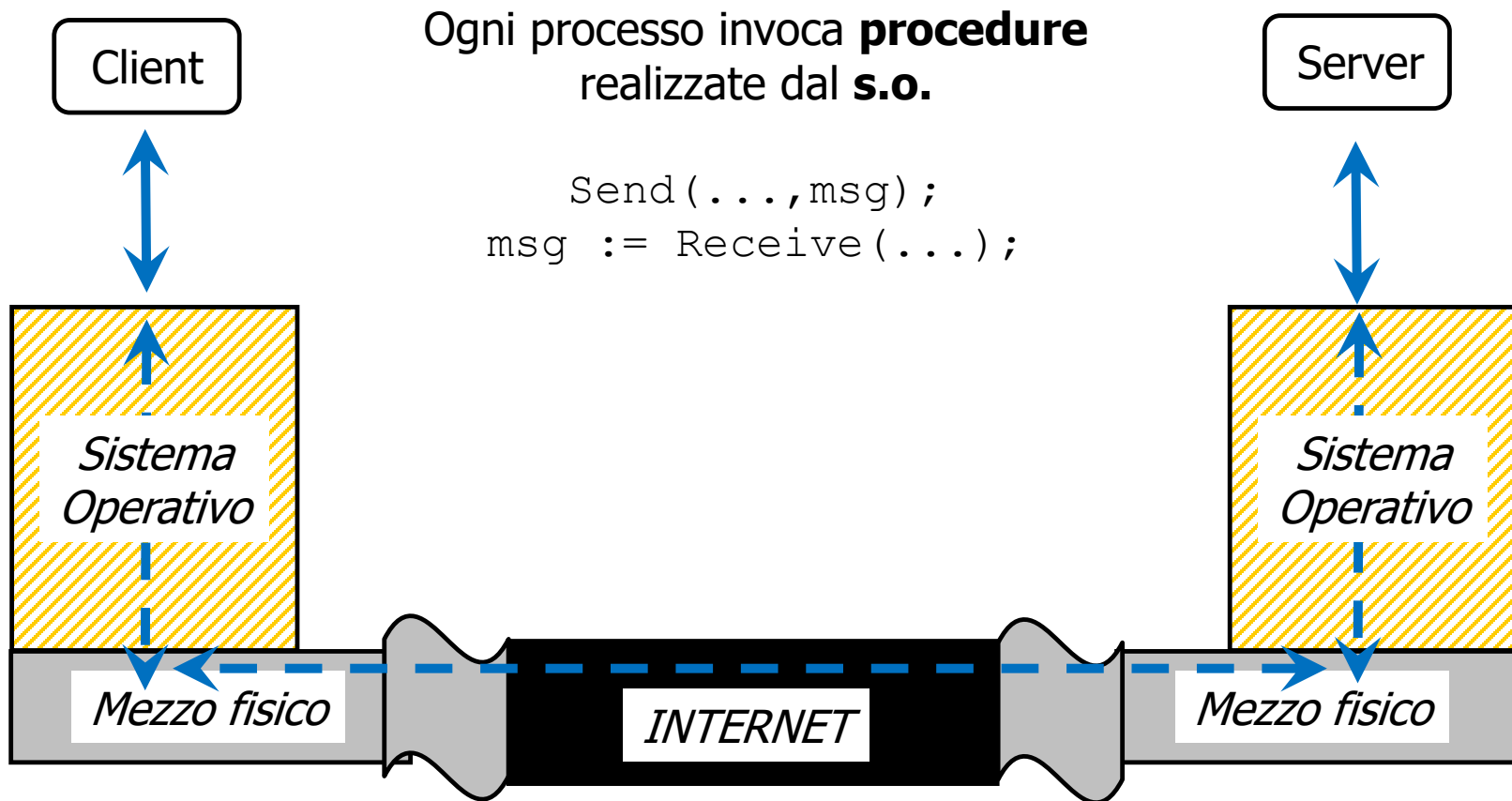
# Linguaggio Comune



# Comunicazione tra Processi



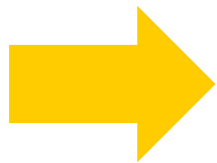
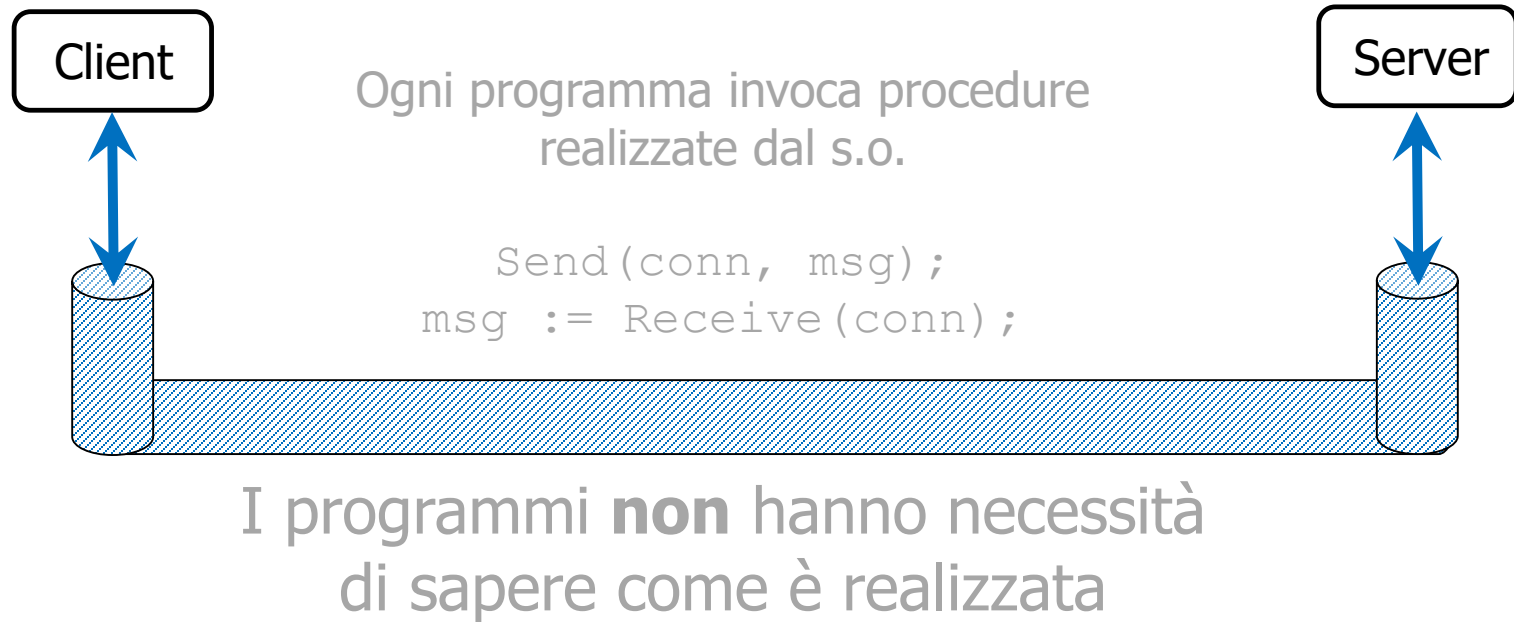
# Comunicazione Fisica



# Comunicazione Logica (I)



# Comunicazione Logica (II)



Come si **usa**

(interfaccia)

~~Come **funziona**~~

~~(implementazione)~~

# Struttura di un Client (I)

1. Determina **identificatore** del server  
(*vedremo come fa ...*)
2. ...

Client

Server

# Struttura di un Client (II)

1. Determina **identificatore** del server  
(vedremo come fa ...)
2. Apre **connessione** ( $\approx$  "tubo" ) verso identificatore server
3. Esegue ciclo:
  - a) Trasmette Request
  - b) Riceve Response
4. Chiude connessione



# Struttura di un Server (I)

1. Sceglie il proprio **identificatore univoco**  
(vedremo come fa...)
2. Attende richieste di apertura connessione "a quell'id."
3. ...

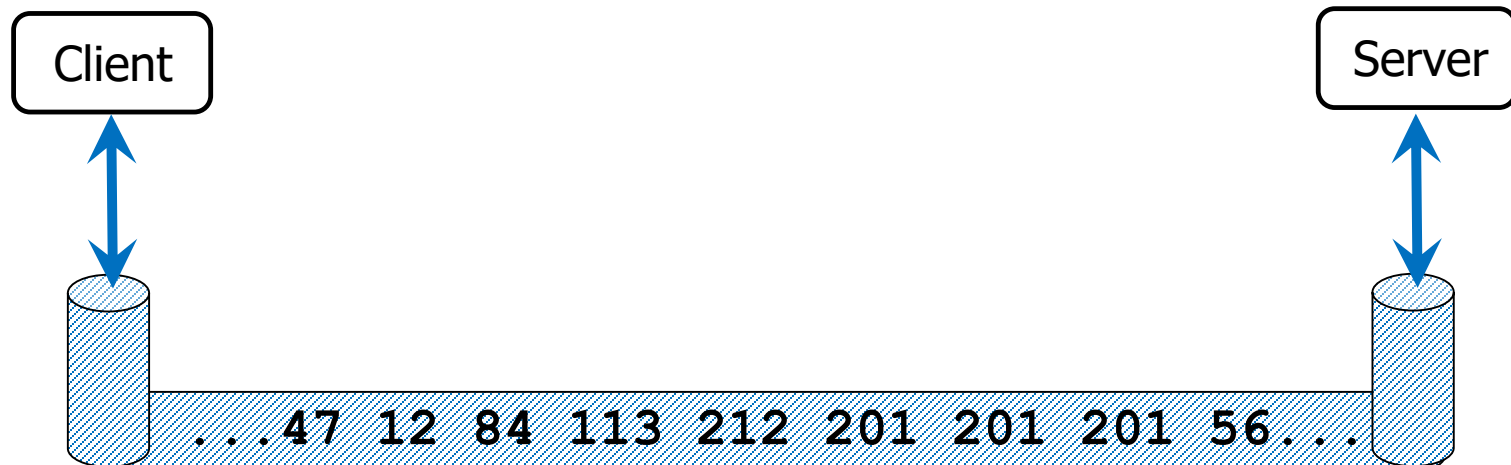


# Struttura di un Server (II)

1. Sceglie il proprio **identificatore univoco**  
(vedremo come fa...)
2. Attende richieste di apertura connessione "a quell'id."
3. Su ogni connessione, esegue ciclo:
  - a) Riceve Request
  - b) Elabora
  - c) Trasmette Response



# Tipo informazioni trasportate



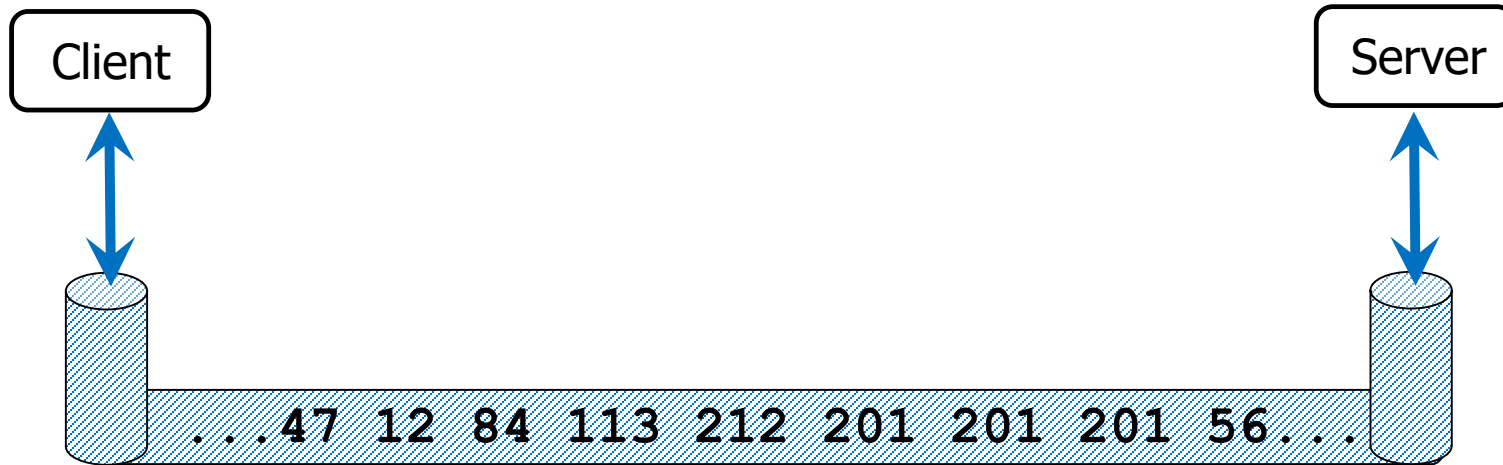
- ❑ Una connessione trasporta solo **sequenze di byte**
- ❑ 1 byte (8 bit) rappresenta un **numero naturale** [0, 255]

# Openness



- ❑ Client e Server possono essere sviluppati:
  - ❑ Dalla **stessa** organizzazione
    - ❑ Skype
    - ❑ Whatsapp
    - ❑ Pochissimi altri casi
  - ❑ Da organizzazioni **diverse**, in luoghi e tempi **diversi**
    - ❑ Caso enormemente più comune

# Hmmm...



- ❑ *Sviluppati da organizzazioni **diverse***
- ❑ ***Come fanno a capirsi???***



# Esempio: Web

- ❑ Web client (browser) collegato con Web server
- ❑ Sviluppati da organizzazioni diverse
- ❑ Client desidera il file index.html

## ❑ Problemi:

- ❑ *Cosa deve contenere una richiesta ?*
- ❑ *La risposta contiene quello che ho chiesto o è un messaggio di errore ?*
- ❑ ...
- ❑ *Prima di inviare una richiesta devo identificarmi ?*
- ❑ *Come mi identifico ?*
- ❑ ...



# Protocollo

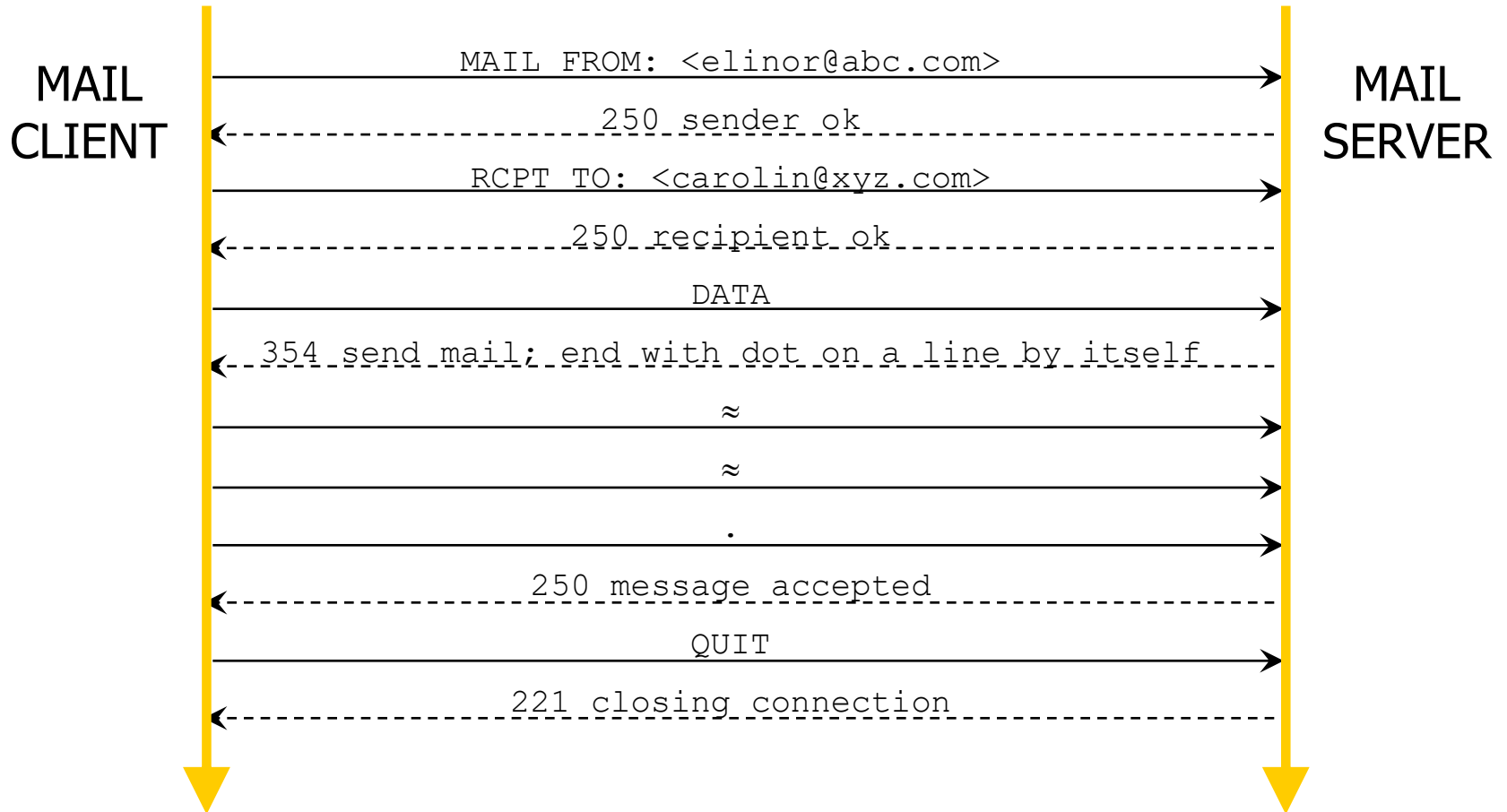
- ❑ Regole note **a priori**
- ❑ Necessarie per permettere alle estremità “di capirsi”

- ❑ Sintassi dei messaggi (struttura)
- ❑ Semantica dei campi (significato)
- ❑ “Quando” si trasmette

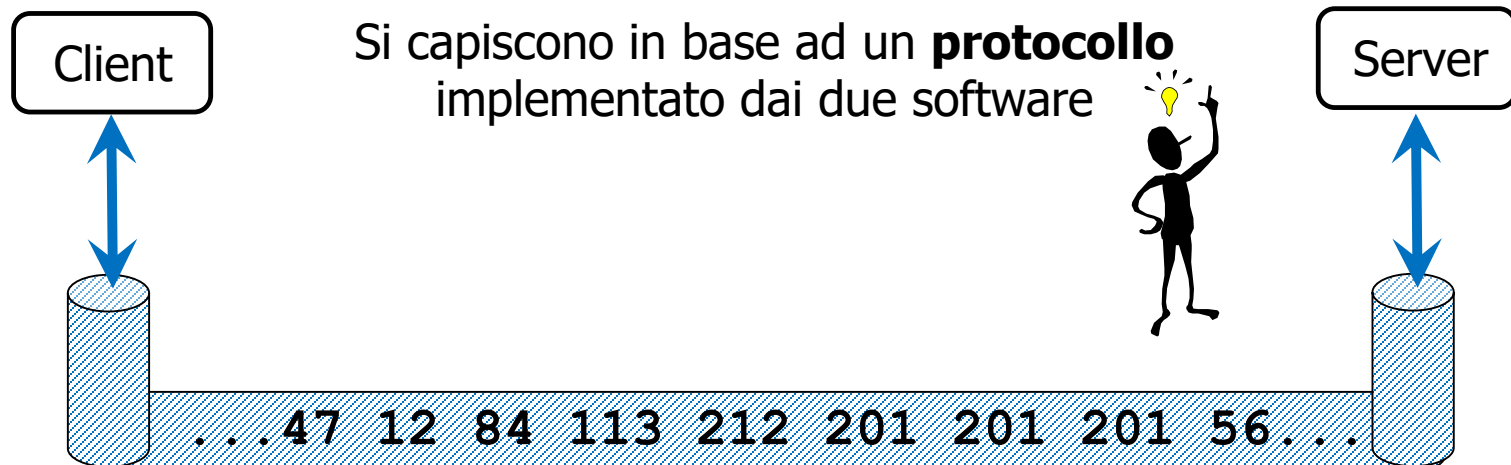
- ❑ I processi che partecipano in una applicazione devono implementare il **protocollo** di quell'applicazione



# Esempio: invio e-mail



# Quindi...



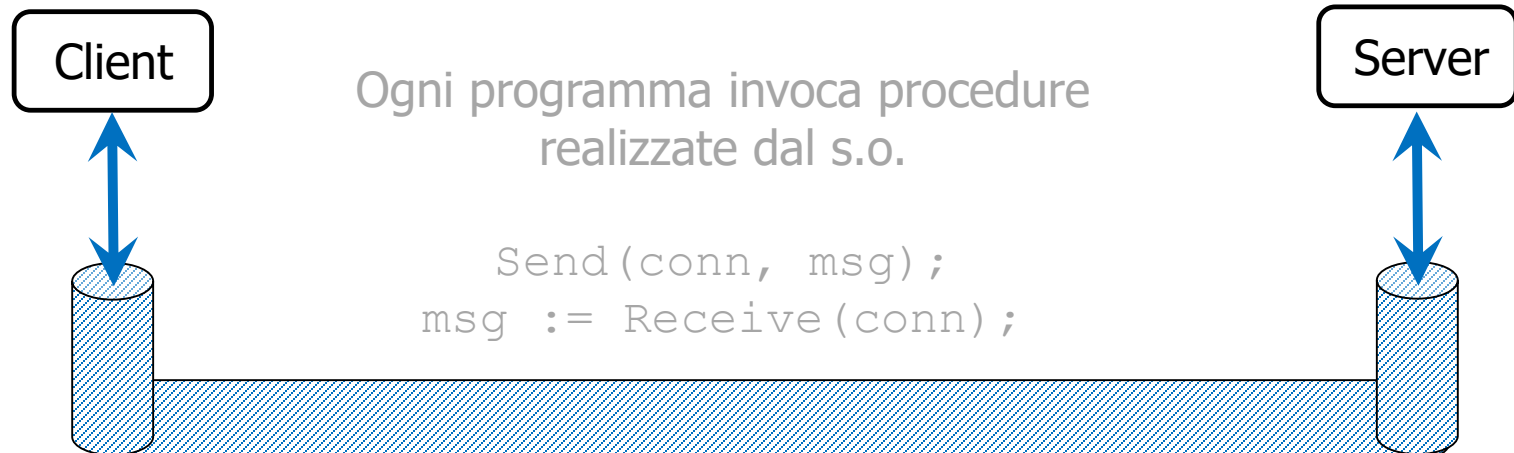
- ❑ Gestiti da Internet Engineering Task Force (IETF)
- ❑ Descritti in documenti chiamati Request For Comments (RFC)
- ❑ **Chiunque** può sviluppare Client e/o Server
- ❑ Vedi sito del corso

# Osservazione



- ❑ Alcuni protocolli (pochissimi) sono:
  - ❑ Mantenuti **riservati**
  - e/o
  - ❑ Coperti da **copyright**
- ❑ Caso importante: Skype
  
- ~~❑ **Chiunque** può sviluppare Client e/o Server~~
- ❑ Problemi tecnici e/o vincoli legali per lo sviluppo di Client e Server

# "Come funziona": POCHISSIME nozioni



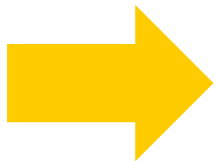
I programmi **non** hanno necessità  
di sapere come è realizzata

Come si **usa**

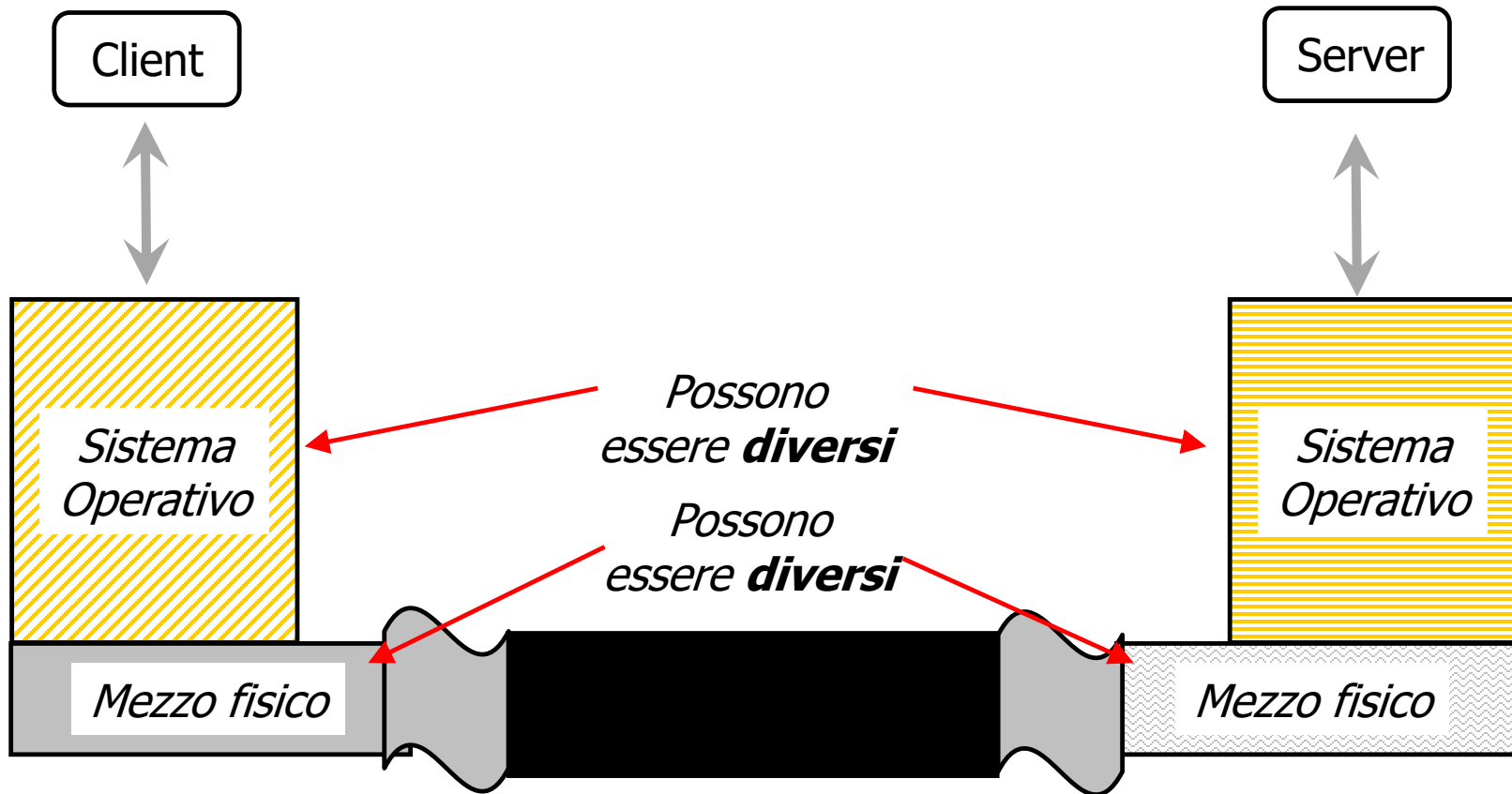
(interfaccia)

Come **funziona**

(implementazione)



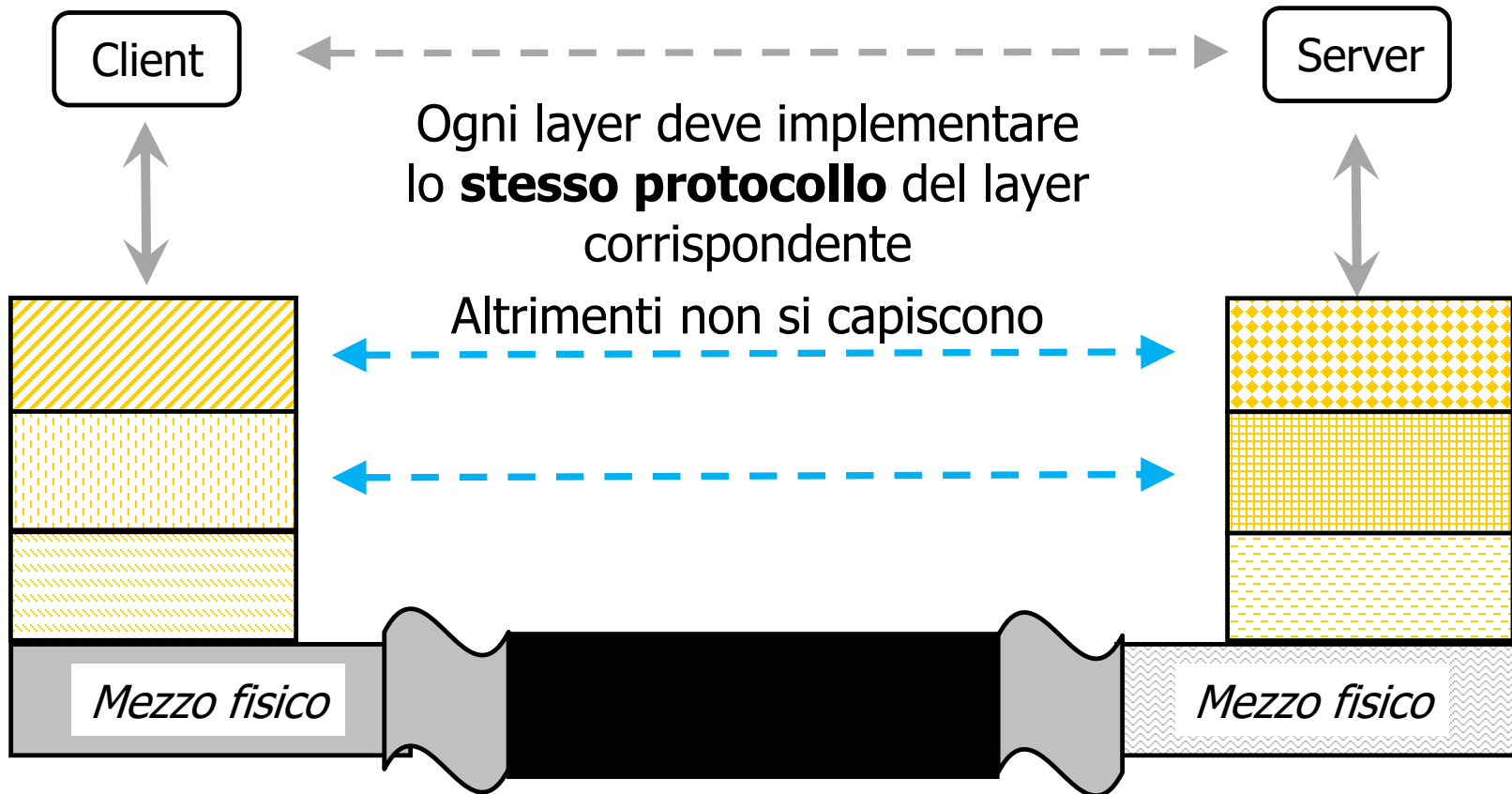
# Eterogeneità



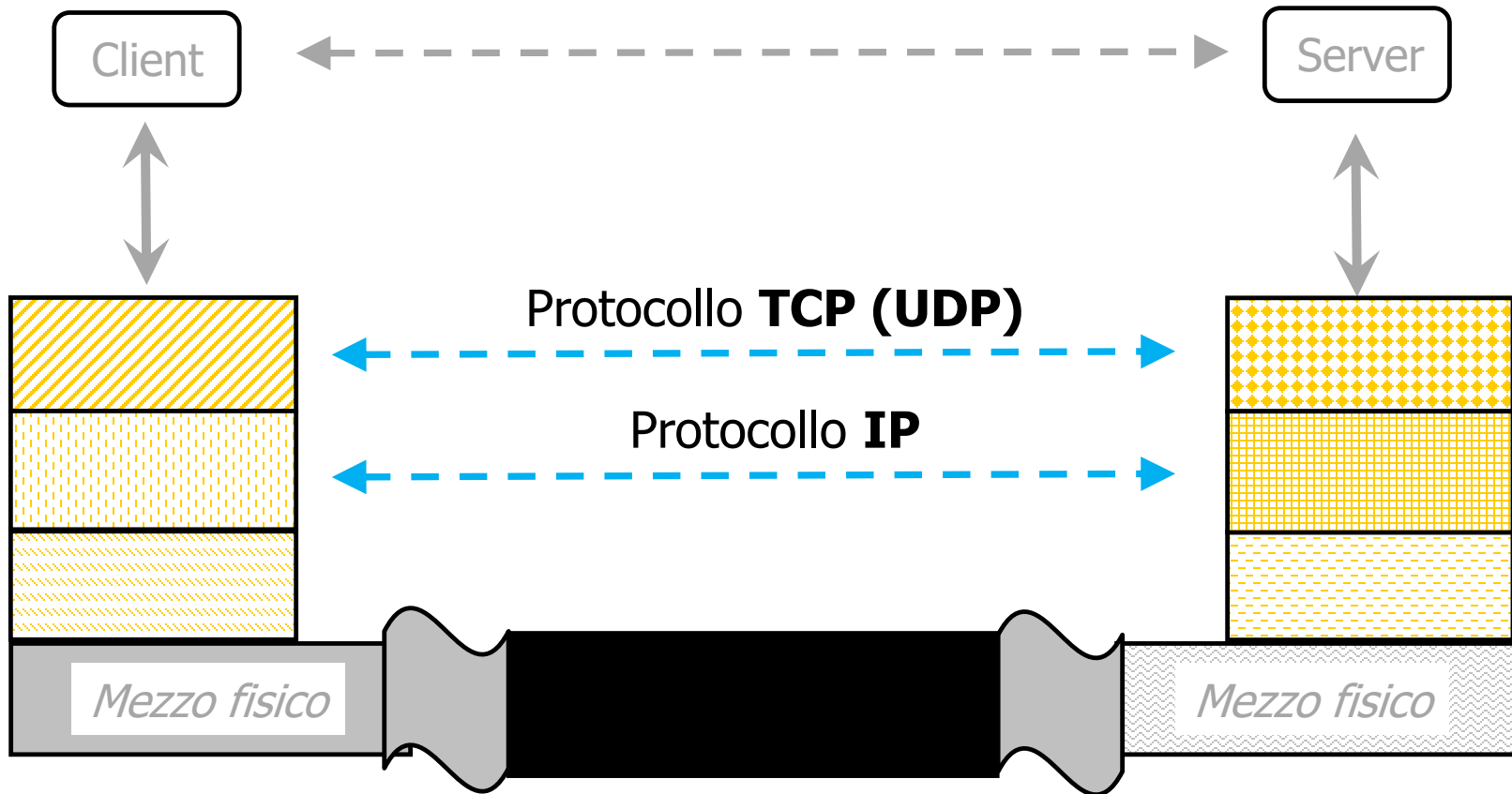
# Layering



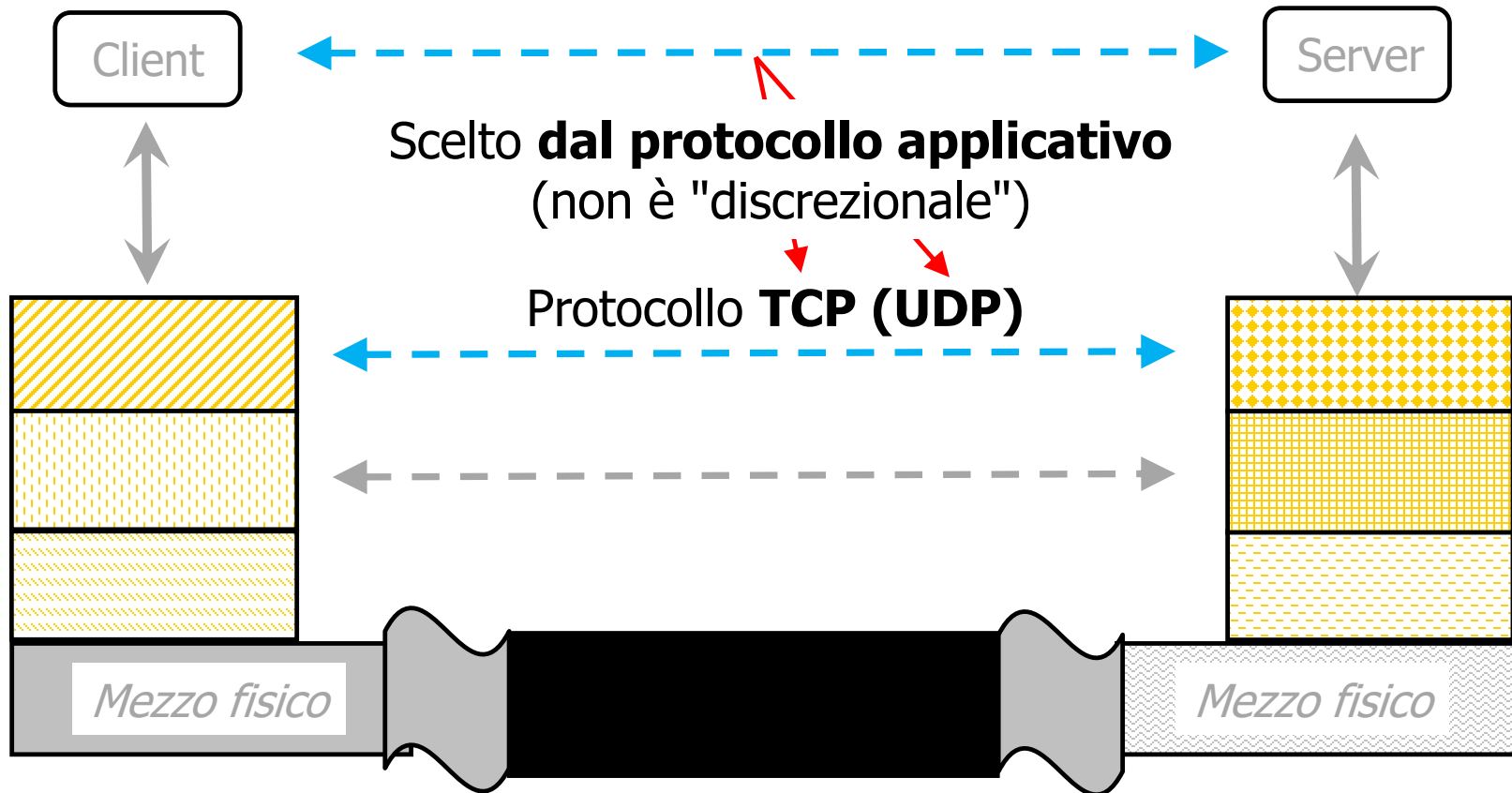
# Sforzo di fantasia



# TCP (UDP) e IP



# TCP o UDP ?



# TCP o UDP?

## Questo corso



- ❑ **"Tutto su TCP"**

- ❑ Tutti i protocolli applicativi di nostro interesse sono su TCP
- ❑ Unica eccezione: DNS
  - ❑ Può funzionare sia su TCP sia su UDP
  - ❑ In pratica è "sempre" su UDP
  - ❑ Noi assumiamo che sia su TCP

# Applicativi TCP



# Hmmm...



# Indirizzo TCP: Identificatore univoco

- ❑ Ogni **processo** collegato a Internet è **identificato univocamente** dal suo **indirizzo TCP**
- ❑ Indirizzo TCP: < **indirizzo IP, port number** >
- ❑ Indirizzo IP:
  - ❑ Identifica il **nodo** su cui è in esecuzione il processo
  - ❑ Naturale su 32 bit  $[0, 2^{32}-1] \approx [0, 4 * 10^9]$
- ❑ Port number:
  - ❑ Identifica il **processo** su quel nodo
  - ❑ Naturale su 16 bit  $[0, 2^{16}-1] = [0, 65535]$

*(parte ultra-semplificata)*

# Indirizzo TCP: Assegnato o Scelto?



□ Indirizzo TCP: < **indirizzo IP**, **port number** >

□ Indirizzo IP:

□ **Assegnato** al nodo (non è scelto)

□ Port number:

□ **Scelto** dai processi **server**

□ **Assegnato** ai processi **client**

*(parte ultra-semplificata)*

# Indirizzo IP: Notazione

- Indirizzo IP: Numero naturale su 32 bit  
 $[0, 2^{32}-1] \approx [0, 4 * 10^9]$
- Per comodità, si esprime in **dotted decimal notation**
  - 32 bit = 4 byte  $\Rightarrow$  4 naturali  $[0, 2^8-1] = [0, 255]$

| 32-bit Binary Number |          |          |          | Equivalent Dotted Decimal |
|----------------------|----------|----------|----------|---------------------------|
| 10000001             | 00110100 | 00000110 | 00000000 | 129 . 52 . 6 . 0          |
| 11000000             | 00000101 | 00110000 | 00000011 | 192 . 5 . 48 . 3          |
| 00001010             | 00000010 | 00000000 | 00100101 | 10 . 2 . 0 . 37           |
| 10000000             | 00001010 | 00000010 | 00000011 | 128 . 10 . 2 . 3          |
| 10000000             | 10000000 | 11111111 | 00000000 | 128 . 128 . 255 . 0       |

# Indirizzo IP vs Posizione Fisica (Geografica)

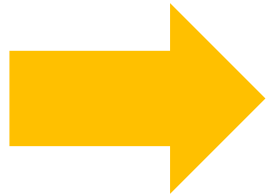
- ❑ E' un **identificatore**

- ❑ **Non** è un **localizzatore**

- ❑ Sintesi grossolana (solo per avere un'idea):

1. Indirizzo IP → Posizione geografica molto approssimata (spesso con grande errore)
2. Nodi "molto vicini" possono avere indirizzi IP "molto simili" (differenza solo byte più a destra)
3. Nodo si sposta "di molti metri" ⇒ Deve cambiare indirizzo IP

# Struttura di un Server (REMININD)

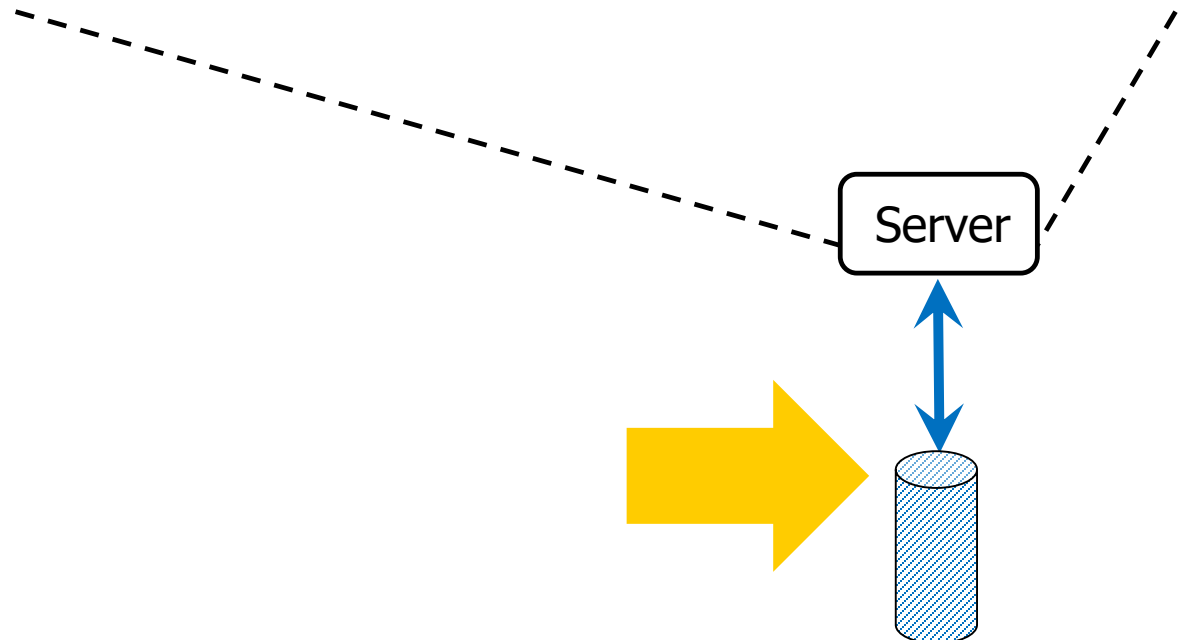


1. Sceglie il proprio **identificatore univoco** (*vedremo come fa...*)
2. Attende richieste di apertura connessione "a quell'id."
3. Su ogni connessione, esegue ciclo:
  - a) Riceve Request
  - b) Elabora
  - c) Trasmette Response



# Server: Scelta indirizzo TCP

1. Sceglie il proprio **identificatore univoco**
  - ☐ **Indirizzo IP** del proprio nodo
  - ☐ **Port number** specificato dal protocollo
2. ...

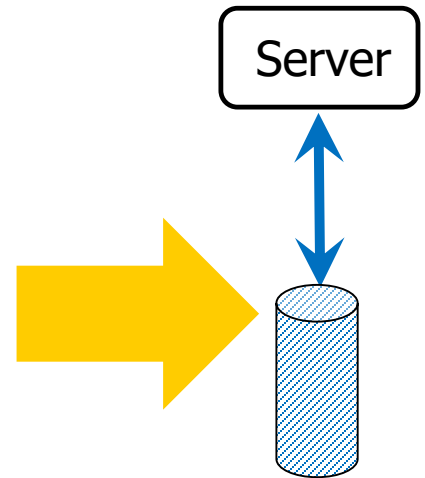


# port number: Server

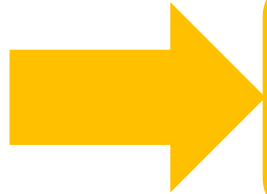
1. Scegliete il proprio **identificatore univoco**
  - ☐ **Indirizzo IP** del proprio nodo
  - ☐ **Port number** specificato dal protocollo
2. ...

|                                                |     |
|------------------------------------------------|-----|
| <input type="checkbox"/> Invio e-mail (SMTP)   | 25  |
| <input type="checkbox"/> Prelievo e-mail (POP) | 110 |
| <input type="checkbox"/> Web (HTTP)            | 80  |
| <input type="checkbox"/> Web crittato (HTTPS)  | 443 |
| <input type="checkbox"/> ...                   |     |

- ☐ SE portNumber è **già usato** da un altro processo
- ☐ ALLORA sistema operativo rifiuta



# Struttura di un Client (REMINDE)

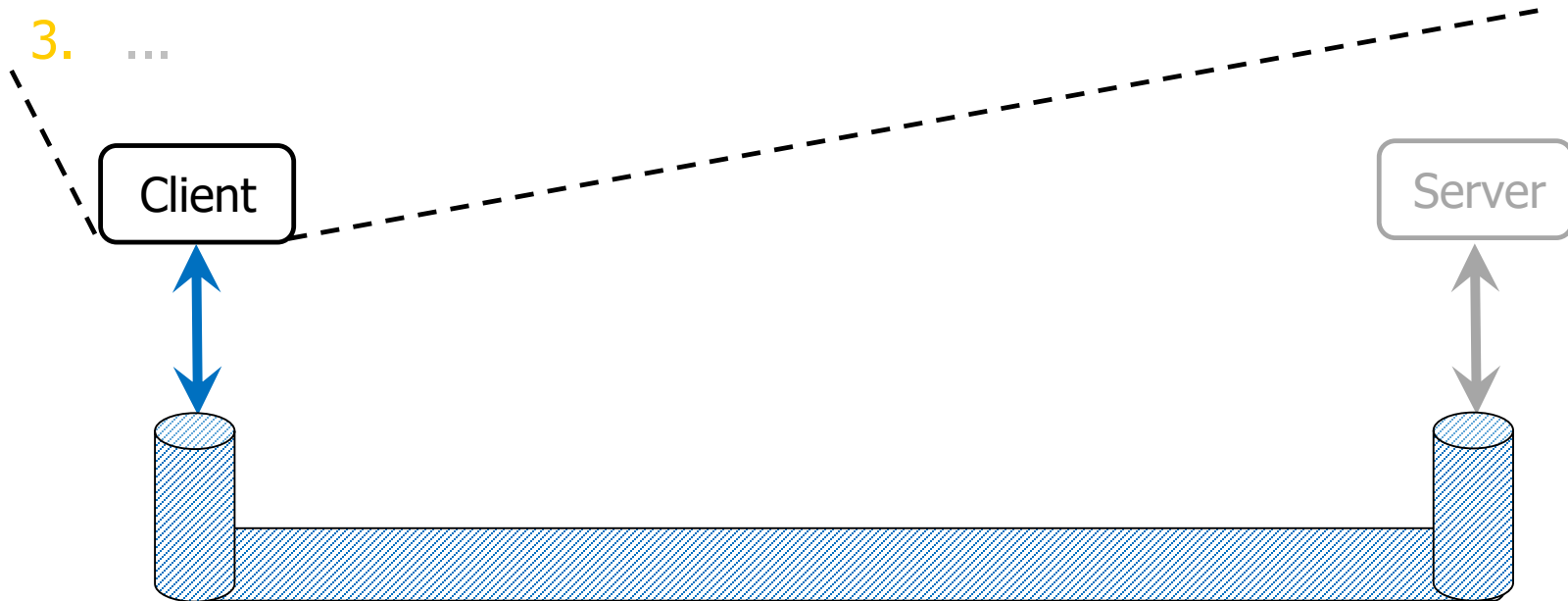


1. Determina **identificatore** del server  
(vedremo come fa ...)
2. Apre **connessione** ( $\approx$  "tubo" ) verso identificatore server
3. Esegue ciclo:
  - a) Trasmette Request
  - b) Riceve Response
4. Chiude connessione



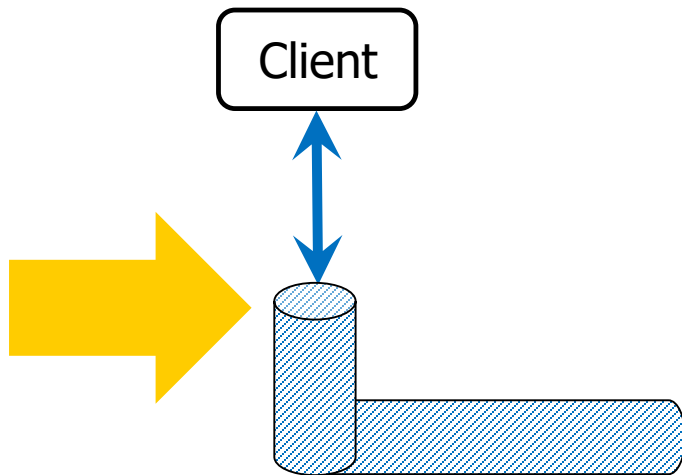
# Client: Scelta indirizzo TCP DEL SERVER

1. Determina **identificatore del server**
  - ❑ **Indirizzo IP** con procedimento che dipende dal protocollo
  - ❑ **Port number** specificato dal protocollo
2. Apre **connessione** ( $\approx$  "tubo" ) **verso identificatore server**
3. ...



# port number: Client

- ❑ **Scelto** dal sistema operativo del client **sul momento**
- ❑ **Diverso** da ogni altro port number attualmente usato sul nodo
- ❑ **Superiore a 1024**

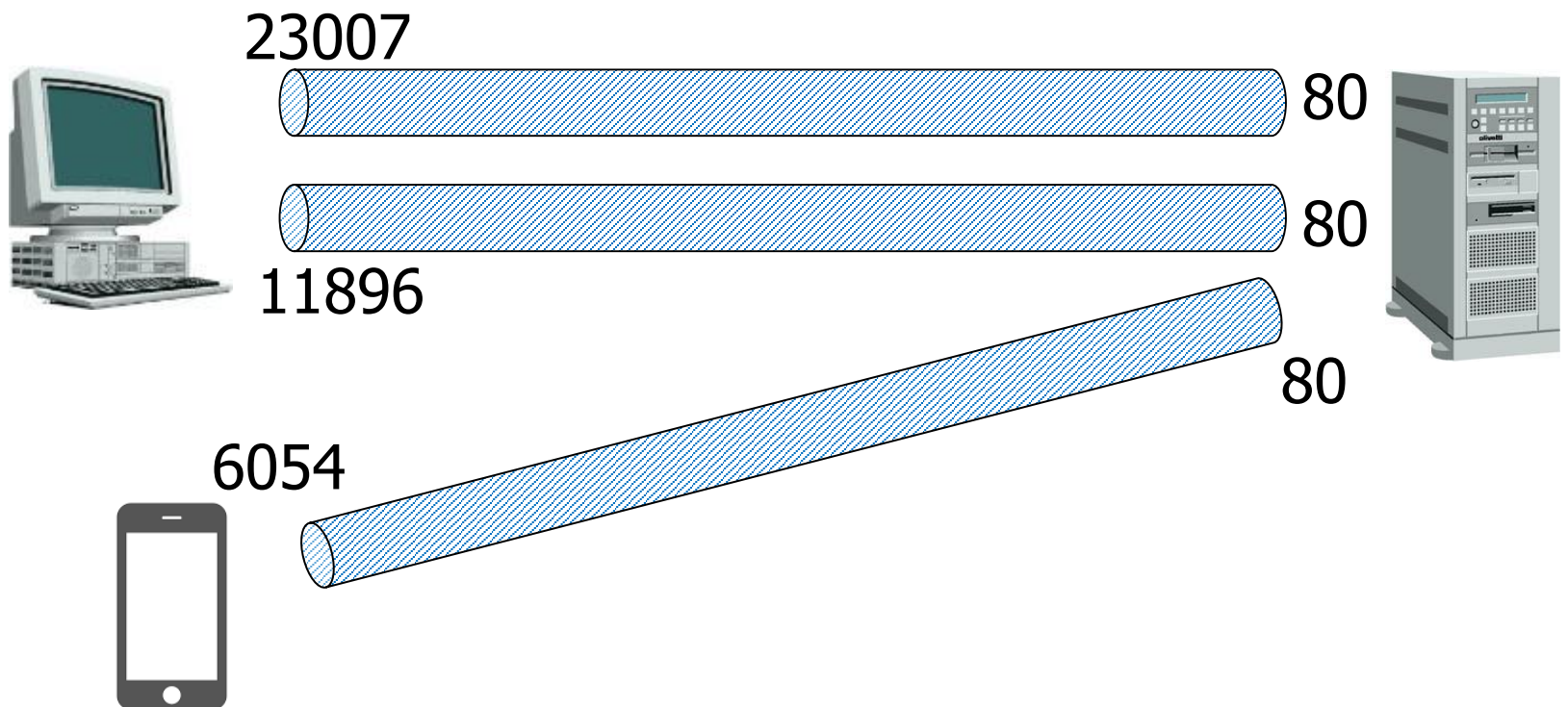


# Importante

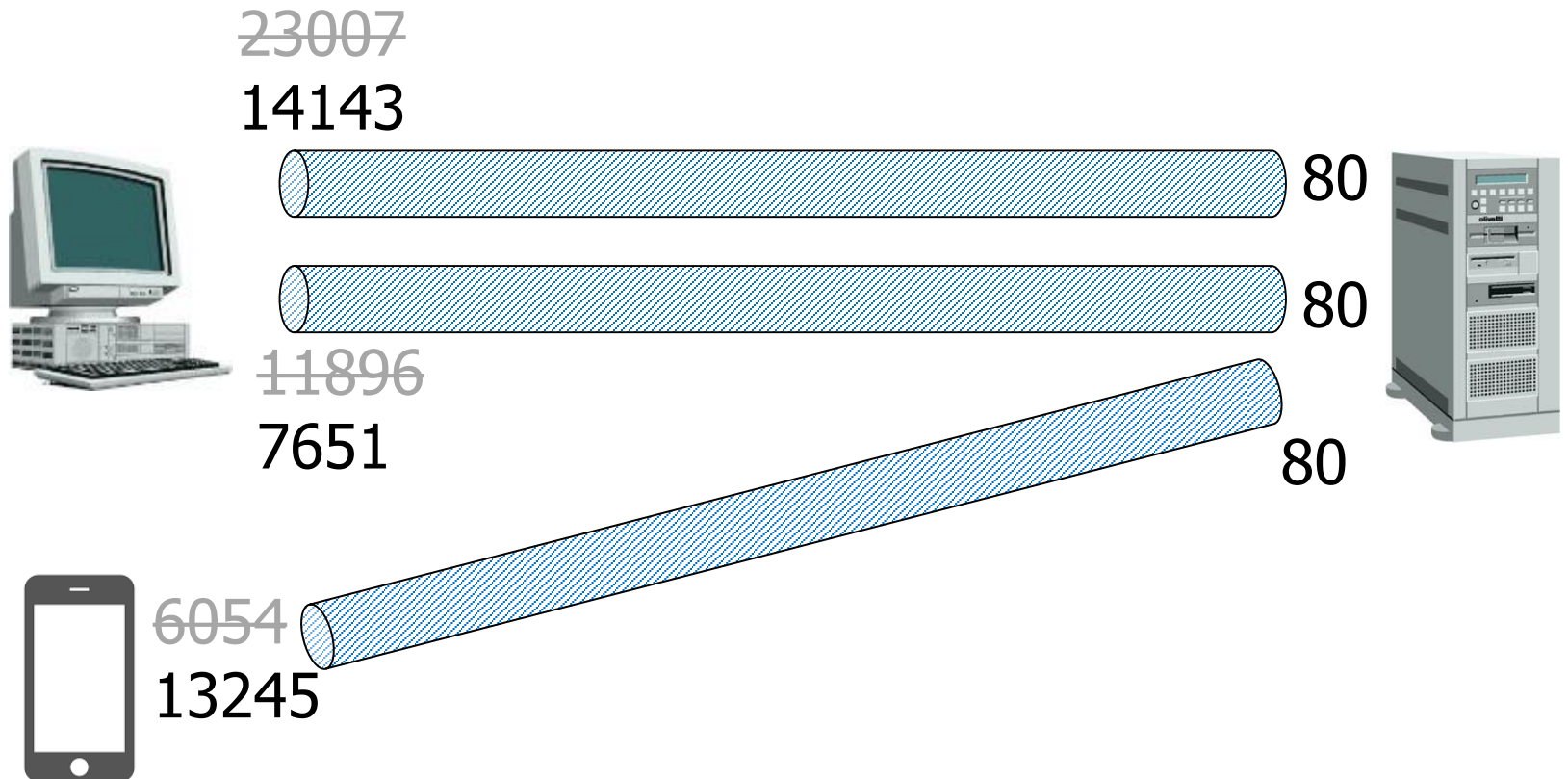


- ❑ port number Server:  
**Scelto** dal server in base al **protocollo**
- ❑ port number Client:  
**Scelto** dal sistema operativo del client **sul momento**
  
- ❑ port number Server: **"non cambia mai"**
- ❑ port number Client: **"cambia ad ogni esecuzione"**

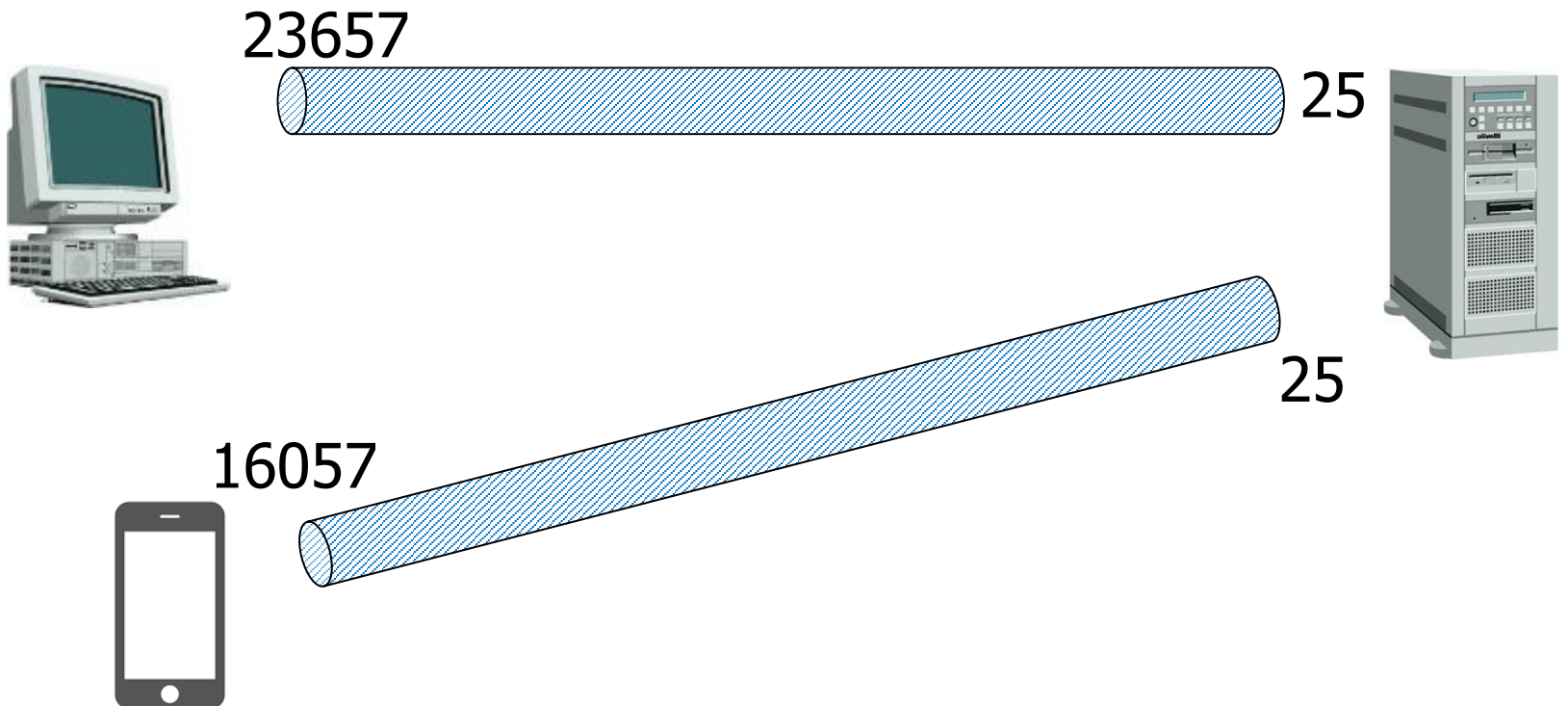
# Esempio: Web Server



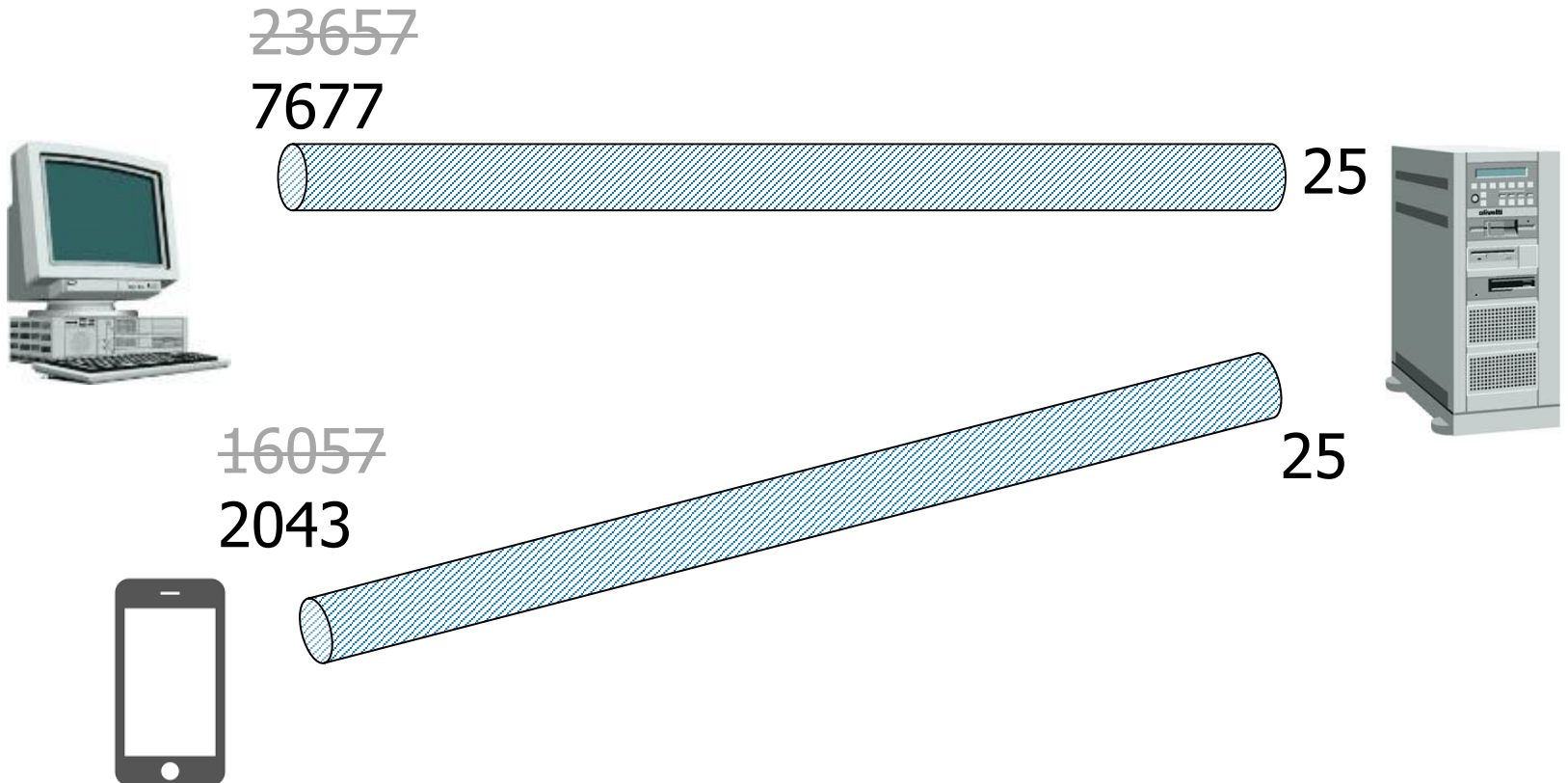
# Esempio: Web Server ("il giorno dopo")



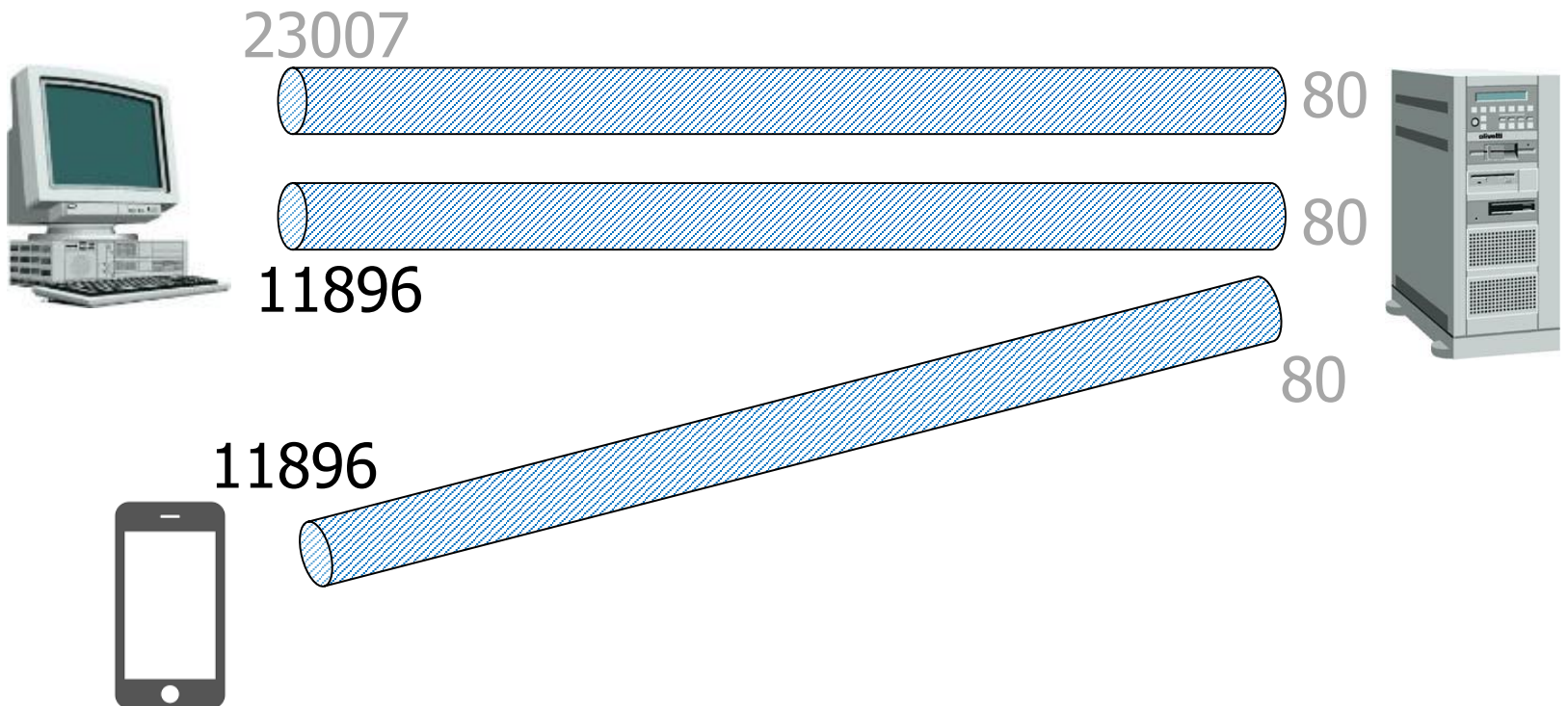
# Esempio: Mail Server (invio)



# Esempio: Mail Server (invio) ("il giorno dopo")



# Possibile?



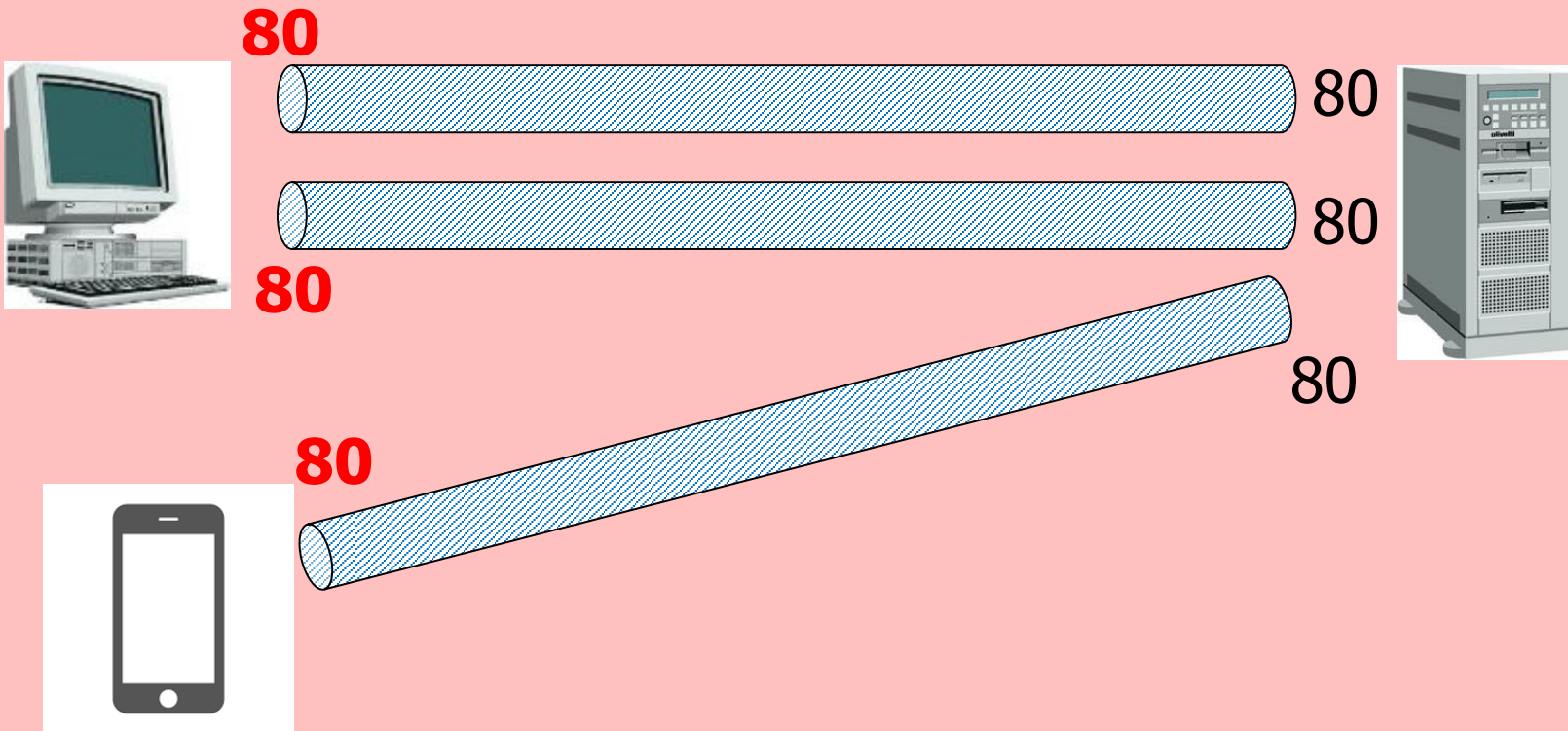
# NOTA BENE



- ❑ Alcune slide sono intitolate "**BOCCIATURA AUTOMATICA**"
- ❑ **NON** esiste nessun tipo di "BOCCIATURA AUTOMATICA"
- ❑ Uso quel titolo **solo** per evidenziare errori:
  - ❑ Relativamente comuni
  - ❑ **Molto gravi**
- ❑ **NON** imparate quegli errori **a memoria**
- ❑ Cercate di capire **perché** sono gravi errori

# Bocciatura automatica

*Indicare il numero di porta lato client*



# Bocciatura (quasi) automatica

*Indicare il numero di porta lato client*



**17120**



80



80

**17120**



# Esempio da traffico reale (I)

- ❑ Browser si collega con vari "web server"
- ❑ Search "github chris sanders pcap"
- ❑ File `http_espn.pcap`
- ❑ Elenco connessioni TCP
- ❑ IP-browser = sempre lo stesso: 172.16.0.122
- ❑ Port-browser = "cambia in ogni connessione, alto"
- ❑ IP-WS = vari indirizzi IP
- ❑ Port-WS = sempre lo stesso: 80

# Esempio da traffico reale (II)

Browser

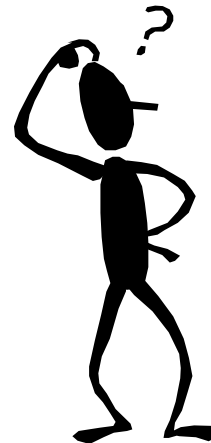
| Address A    | Port A | Address B       | Port B |
|--------------|--------|-----------------|--------|
| 172.16.0.122 | 52166  | 199.181.132.250 | 80     |
| 172.16.0.122 | 44955  | 68.71.208.11    | 80     |
| 172.16.0.122 | 41834  | 205.234.218.129 | 80     |
| 172.16.0.122 | 41835  | 205.234.218.129 | 80     |
| 172.16.0.122 | 41161  | 205.234.218.82  | 80     |
| 172.16.0.122 | 41162  | 205.234.218.82  | 80     |
| 172.16.0.122 | 41838  | 205.234.218.129 | 80     |
| 172.16.0.122 | 41839  | 205.234.218.129 | 80     |
| 172.16.0.122 | 58485  | 63.85.36.8      | 80     |
| 172.16.0.122 | 33720  | 63.85.36.72     | 80     |
| 172.16.0.122 | 41842  | 205.234.218.129 | 80     |
| 172.16.0.122 | 35993  | 205.234.218.112 | 80     |
| 172.16.0.122 | 41844  | 205.234.218.129 | 80     |
| 172.16.0.122 | 33987  | 68.71.209.72    | 80     |
| 172.16.0.122 | 35996  | 205.234.218.112 | 80     |
| 172.16.0.122 | 41847  | 205.234.218.129 | 80     |
| 172.16.0.122 | 41848  | 205.234.218.129 | 80     |
| 172.16.0.122 | 33728  | 63.85.36.72     | 80     |
| 172.16.0.122 | 36000  | 205.234.218.112 | 80     |
| 172.16.0.122 | 33122  | 205.234.218.67  | 80     |
| 172.16.0.122 | 53146  | 68.71.208.113   | 80     |
| 172.16.0.122 | 44975  | 68.71.208.11    | 80     |
| 172.16.0.122 | 35135  | 66.235.139.152  | 80     |
| 172.16.0.122 | 55880  | 68.71.208.177   | 80     |
| 172.16.0.122 | 38433  | 63.85.36.9      | 80     |

# Struttura di un Client: Indirizzo IP Server?



# Struttura di un Client: Indirizzo IP Server?

1. Determina **identificatore del server**
  - ❑ **Indirizzo IP** con procedimento che dipende dal protocollo
  - ❑ **Port number** specificato dal protocollo
2. ...



# Struttura di un Client: Indirizzo IP Server



1. Determina **identificatore del server**
  - ❑ **Indirizzo IP** con procedimento che **dipende dal protocollo**
  - ❑ **Port number** specificato dal protocollo
2. ...
  - ❑ Vedremo in dettaglio
  - ❑ Anticipiamo descrizione intuitiva per avere un'idea

# Configurazione: Che significa



- ❑ **Informazioni** che **devono** essere **note a priori** ad un processo / host
- ❑ **Come** si specificano per noi **irrilevante**
  - ❑ Dipende da applicazione, protocollo, sistema operativo
- ❑ **Quali sono** per noi **fondamentale**
  - ❑ Necessario per capire funzionamento Internet

# Configurazione: Ogni nodo Internet



- ☐ ...
- ☐ ...
- ☐ Indirizzo IP di un "server DNS" (IP-DNS)
- ☐ ...

# Browser (HTTP): Indirizzo IP Server

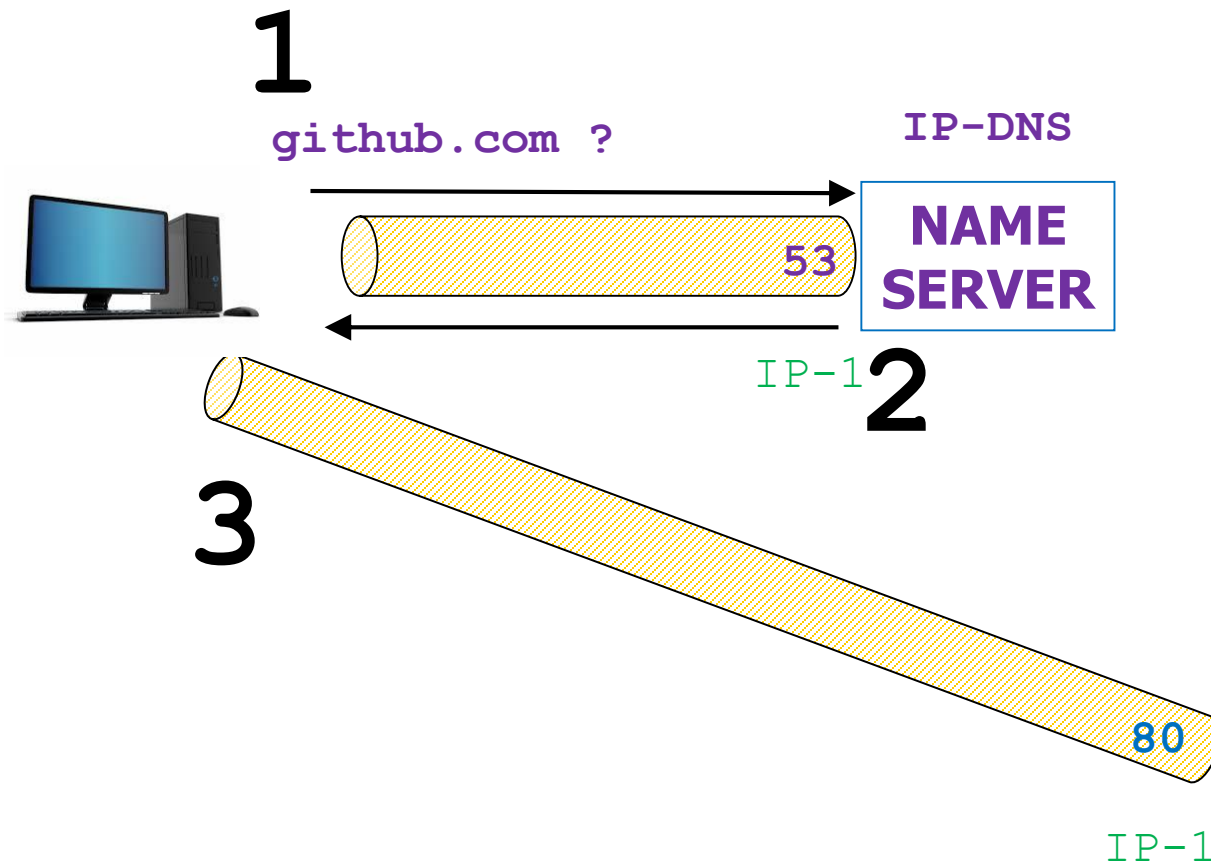
1. "URL della pagina web" → "Nome server"

- ❑ <https://portale.units.it/it/ateneo/chi-siamo>
- ❑ <https://github.com/PacktPublishing/Hands-On-Penetration-Testing-on-Windows>
- ❑ <https://www.ilpost.it/2024/07/09/sonar-podcast-balene/?homepagePosition=3>

2. "Nome server" → IP-server

- ❑ Apre connessione con (**IP-DNS**, 53)
- ❑ Interazione con protocollo DNS:
  - ❑ Invia richiesta "indirizzo IP di [portale.units.it](https://portale.units.it) ?"
  - ❑ Riceve risposta contenente **IP-server**

# Esempio



# Esempio da traffico reale (III)

- ❑ Browser si collega con vari "web server"
- ❑ Search "github chris sanders pcap"
- ❑ File `http_espn.pcap`
- ❑ Elenco connessioni TCP
- ❑ Per ottenere indirizzo IP dei vari web server deve contattare **name server**
- ❑ IP-browser = sempre lo stesso: 172.16.0.122
- ❑ Port-browser = "cambia in ogni connessione, alto"
- ❑ IP-NS = **sempre lo stesso: 4.2.2.1**
- ❑ Port-NS = sempre lo stesso: 53

# Esempio da traffico reale (IV)

Browser

| Address A    | Port A | Address B | Port B |
|--------------|--------|-----------|--------|
| 172.16.0.122 | 56346  | 4.2.2.1   | 53     |
| 172.16.0.122 | 40510  | 4.2.2.1   | 53     |
| 172.16.0.122 | 54903  | 4.2.2.1   | 53     |
| 172.16.0.122 | 40973  | 4.2.2.1   | 53     |
| 172.16.0.122 | 35631  | 4.2.2.1   | 53     |
| 172.16.0.122 | 52325  | 4.2.2.1   | 53     |
| 172.16.0.122 | 56081  | 4.2.2.1   | 53     |
| 172.16.0.122 | 56049  | 4.2.2.1   | 53     |
| 172.16.0.122 | 48362  | 4.2.2.1   | 53     |
| 172.16.0.122 | 41251  | 4.2.2.1   | 53     |
| 172.16.0.122 | 40021  | 4.2.2.1   | 53     |
| 172.16.0.122 | 46890  | 4.2.2.1   | 53     |
| 172.16.0.122 | 33399  | 4.2.2.1   | 53     |
| 172.16.0.122 | 48467  | 4.2.2.1   | 53     |

# Client email (SMTP/POP): Indirizzo IP Server

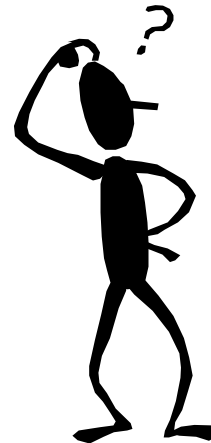


1. "Il proprio indirizzo email" → "Nome server"
  - ❑ bartoli.alberto@units.it → mx.units.it  
(vedremo meglio più avanti nel corso)
  
2. "Nome server" → IP-server
  - ❑ Apre connessione con (**IP-DNS**, 53)
  - ❑ Interazione con protocollo DNS:
    - ❑ Invia richiesta "indirizzo IP di mx.units.it ?"
    - ❑ Riceve risposta contenente IP-server

# Hmmm...

1. "Il proprio indirizzo email" → "Nome server"
  - bartoli.alberto@units.it → mx.units.it  
(vedremo meglio più avanti nel corso)

*Come fa il programma client email  
a conoscere il proprio indirizzo email?*



# Configurazione: Esempi



## ☐ Client email

☐ Configurazione: "Il proprio indirizzo email"

## ☐ Browser

☐ Configurazione: **Nulla**  
(a meno di non navigare tramite "proxy": vedremo)

# Ottenere Indirizzo IP Server



1. Step dipendente dal protocollo
2. "Nome server" → IP-server
  - ☐ **SEMPRE** presente
  - ☐ Basato su DNS
  - ☐ Richiede IP-server in configurazione

# Socket



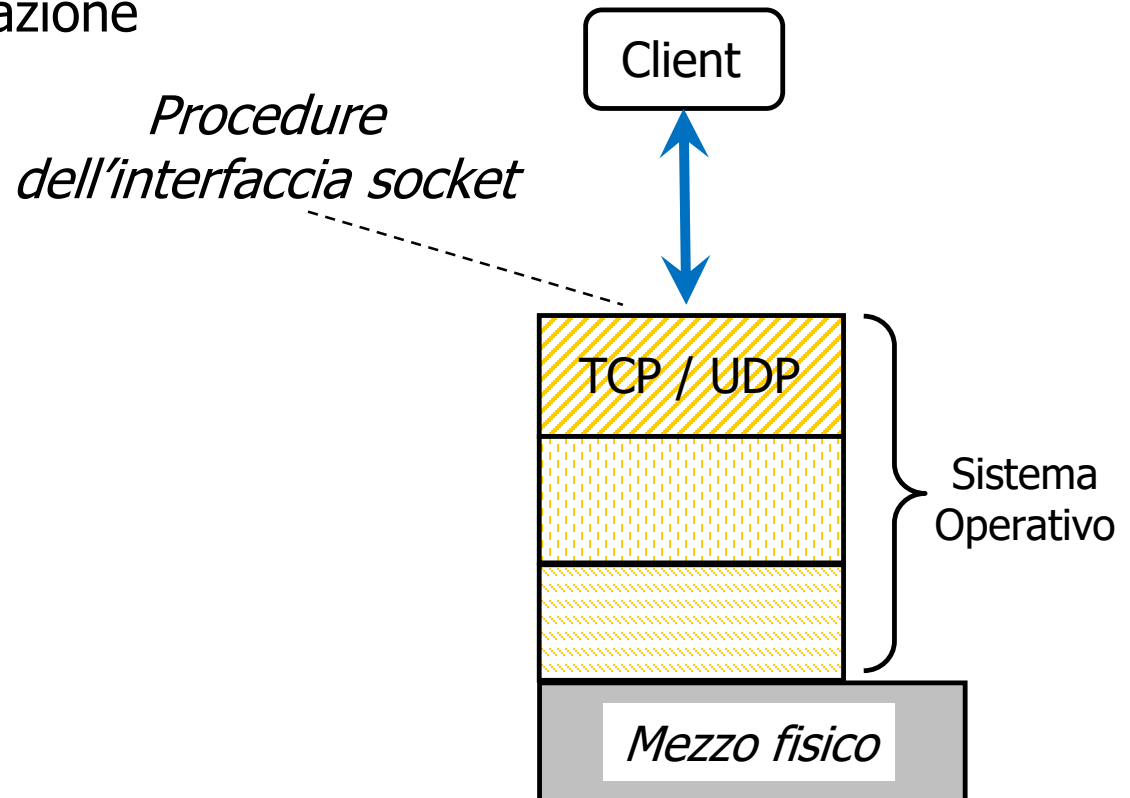
# Quali sono le procedure?



# Socket (I)

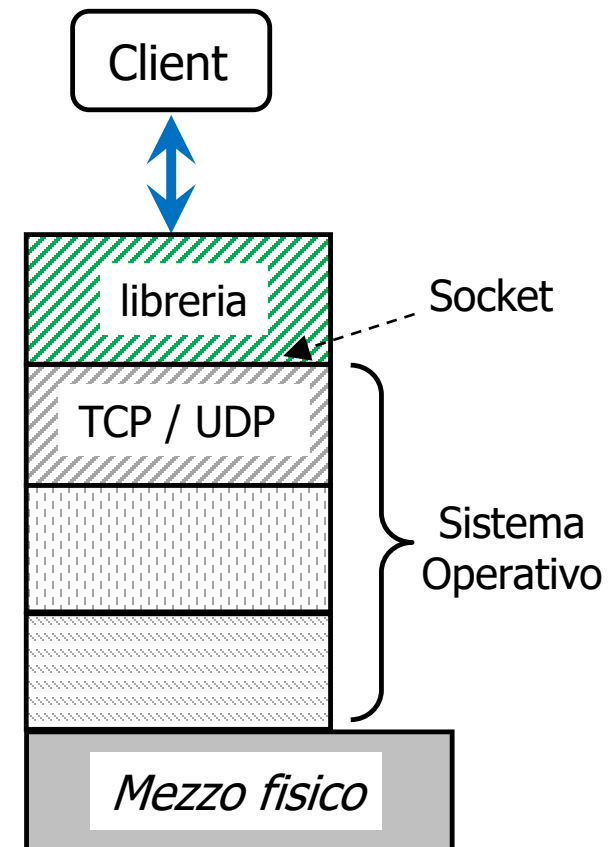
- ❑ TCP - UDP si accedono attraverso un'interfaccia di programmazione chiamata "**socket**"

- ❑ **Quasi**-standard
- ❑ Molto complicata



# Socket (II)

- ❑ Molto spesso i programmi usano **librerie** con un livello di astrazione **più alto**
  - ❑ Insieme di procedure già scritte
- ❑ Hanno interfaccia "**molto più semplice**" della interfaccia socket
- ❑ **Non** standard
- ❑ Dipende (anche) dal linguaggio di programmazione
  - ❑ Java
  - ❑ Python
  - ❑ ...
  - ❑ C
  - ❑ ...



# Socket: Interfaccia TCP (ultra)semplificata (I)

| Primitive                        | Meaning                               |
|----------------------------------|---------------------------------------|
| SOCKET                           | Create a new communication end point  |
| Procedure per aprire connessione |                                       |
| SEND                             | Send some data over the connection    |
| RECEIVE                          | Receive some data from the connection |
| CLOSE                            | Release the connection                |

# Socket: Interfaccia TCP (ultra)semplificata (II)

| Server<br>Client | Primitive | Meaning                                                     |
|------------------|-----------|-------------------------------------------------------------|
|                  | SOCKET    | Create a new communication end point                        |
|                  | BIND      | Attach a local address to a socket                          |
|                  | LISTEN    | Announce willingness to accept connections; give queue size |
|                  | ACCEPT    | Block the caller until a connection attempt arrives         |
|                  | CONNECT   | Actively attempt to establish a connection                  |
|                  | SEND      | Send some data over the connection                          |
|                  | RECEIVE   | Receive some data from the connection                       |
|                  | CLOSE     | Release the connection                                      |

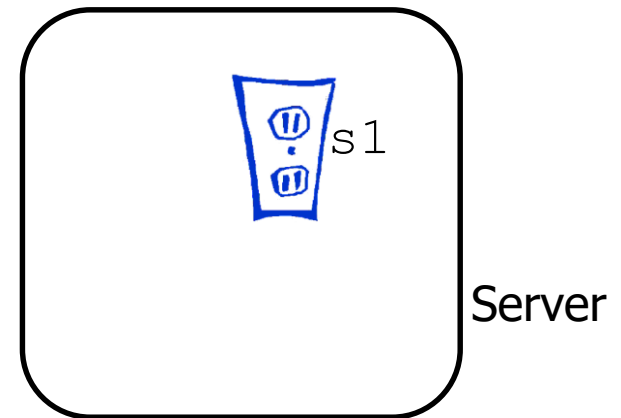
# Struttura di un Server (REMINDE)

1. Sceglie il proprio **identificatore univoco**  
(vedremo come fa...)
2. Attende richieste di apertura connessione "a quell'id."
3. Su ogni connessione, esegue ciclo:
  - a) Riceve Request
  - b) Elabora
  - c) Trasmette Response



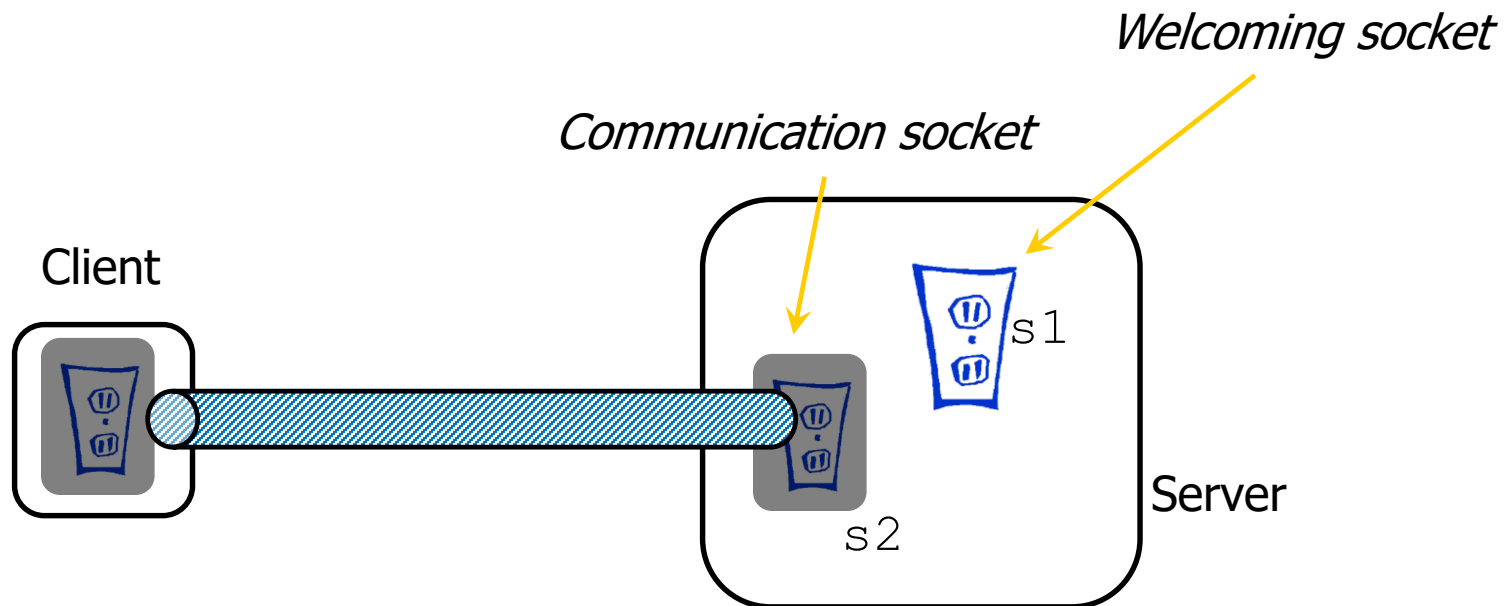
# Server (ultra)semplificato (I)

- ❑ Crea socket  $s1$
- ❑ Sceglie un port number
- ❑ Associa  $s1$  al port number
- ❑ Dichiarare disponibilità a connessioni su  $s1$
- ❑ Attende connessione su  $s1$
- ❑ ...



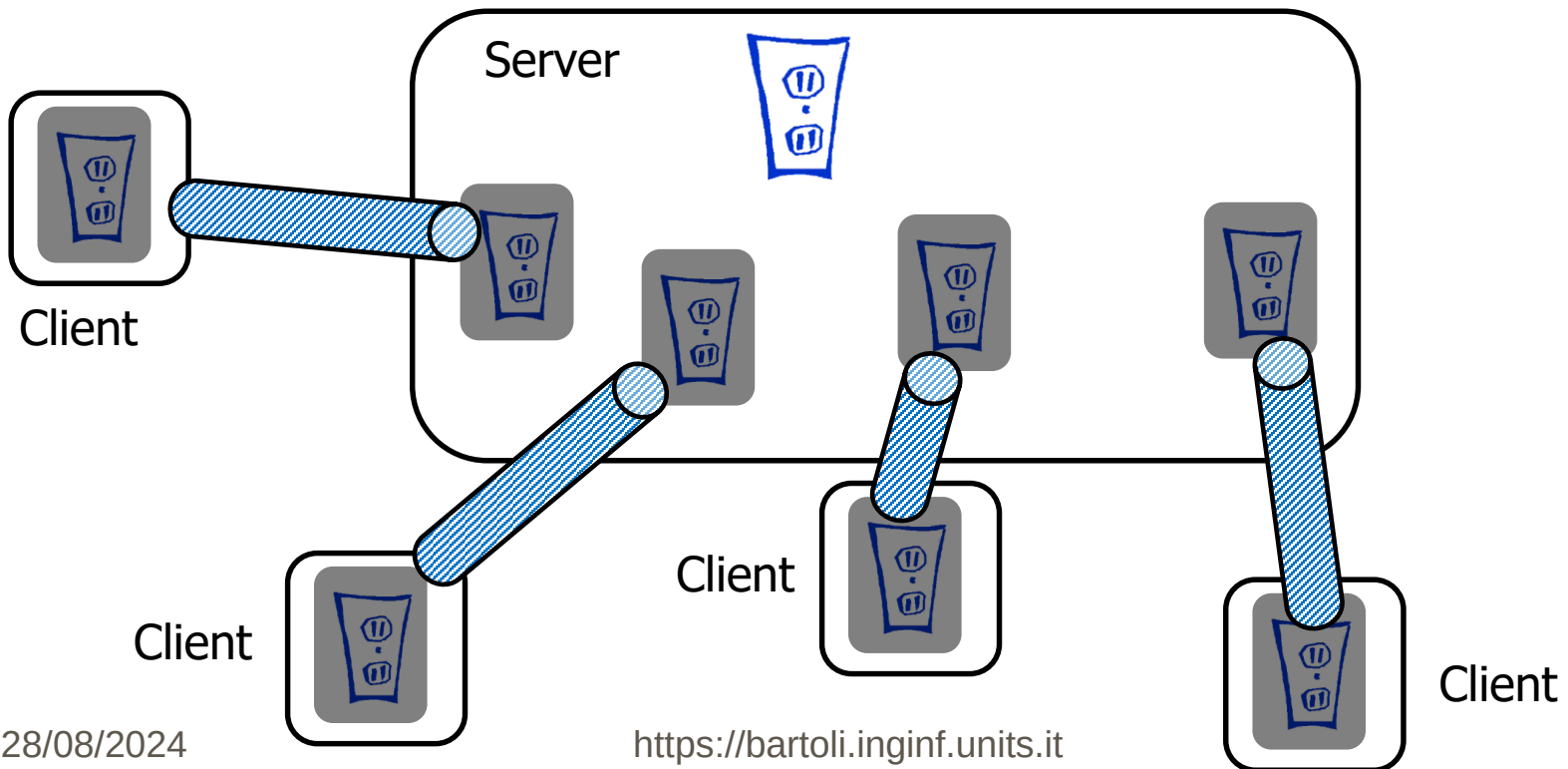
# Server (ultra)semplificato (II)

- ❑ ...
- ❑ Ottiene **un altro** socket  $s_2$  per la connessione appena aperta
- ❑ Comunica su  $s_2$  (in base ad application protocol)



# Server con $K$ client connessi

- 1 welcoming socket
- $K$  communication socket



# Codice (ultra)semplificato

```
s1 := socket(...); ← ☐ Crea un socket s1
bind(s1, portSrv, ...); ← ☐ Sceglie port number e lo associa a s1
listen(s1, ...); ← ☐ Dichiaro disponibilità a connessioni su s1
s2 := accept(s1, ...); ← ☐ Attende su s1 e ottiene nuovo socket s2
```

```
...
msg1 := receive(s2, ...);
...
send(s2, msg2, ...);
...
```

Messaggi definiti nello  
application-layer protocol

# Ultra-super-importante (Uso vs Implementazione)

```
int s1,s2;

s1 := socket(...);
bind(s1,portSrv,...);
listen(s1,...);
s2 := accept(s1,...);
...
msg1 := receive(s2,...);
...
send(s2,msg2,...);
...
```

- ❑ L'applicativo **non** ha necessità di sapere **come sono realizzate** le primitive
- ❑ Basta sapere **come usarle**

# IMPORTANTISSIMO (in generale)



Distinguere tra:

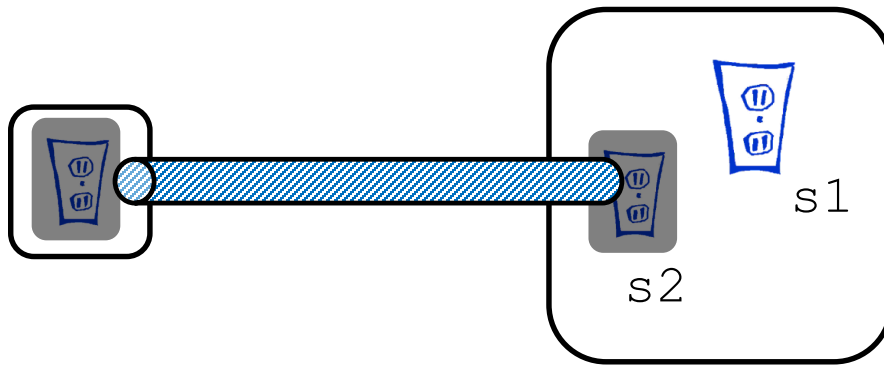
- ☐ **Come si usa** ("INTERFACCIA")
- ☐ **Come funziona** ("IMPLEMENTAZIONE")

**Fondamentale in ogni settore tecnologico**  
(e non solo)

Sforzatevi di pensare **sempre** in questi termini...

# Server:

## Osservazione 1

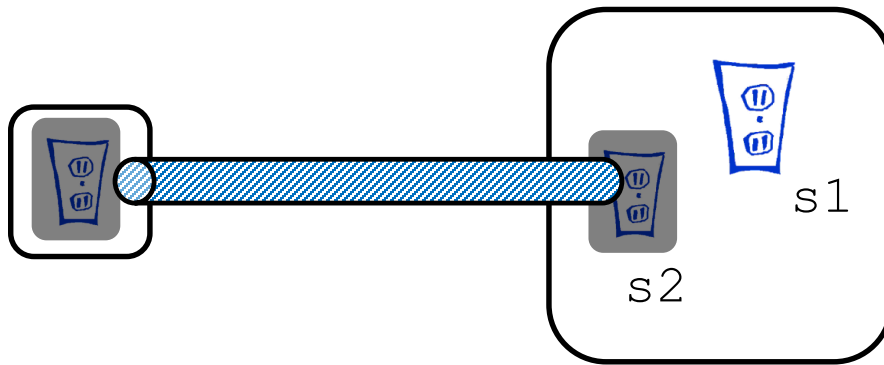


```
int s1, s2;  
  
s1 := socket(...);  
bind(s1, portSrv, ...);  
listen(s1, ...);  
s2 := accept(s1, ...);  
...  
msg1 := receive(s2, ...);  
...  
send(s2, msg2, ...);  
...
```

- ❑ L'applicativo non ha necessità di conoscere  $\langle \text{IP-cln}, \text{port-cln} \rangle$
- ❑ Può conoscerli con altre funzioni
- ❑ Possono servire in casi particolari  
(decido di offrire servizio solo a IP-cln "noti")

# Server:

## Osservazione 2

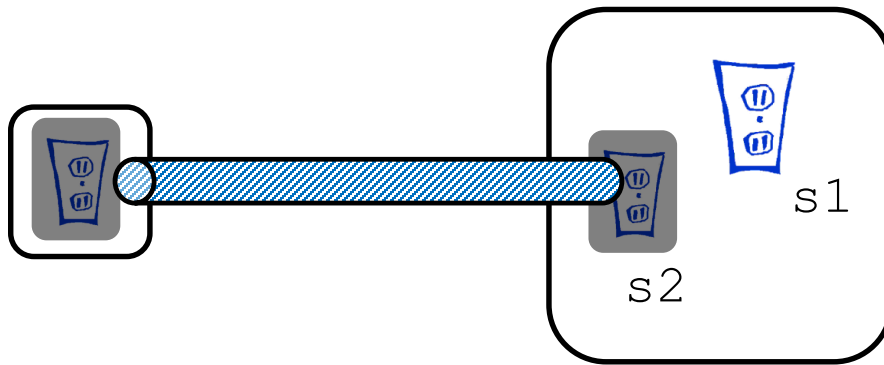


```
int s1, s2;  
  
s1 := socket(...);  
bind(s1, portSrv, ...);  
listen(s1, ...);  
s2 := accept(s1, ...);  
...  
msg1 := receive(s2, ...);  
...  
send(s2, msg2, ...);  
...
```

- ❑ L'applicativo usa il proprio port number **solo** nella `bind()` (**prima** di comunicare con i client)
- ❑ Per comunicare con i client non lo usa

# Server:

## Osservazione 3

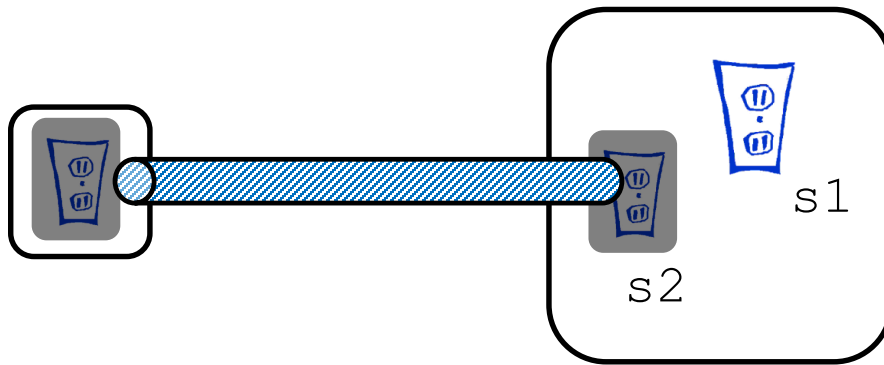


```
int s1,s2;

s1 := socket(...);
bind(s1,portSrv,...);
listen(s1,...);
s2 := accept(s1,...);
...
msg1 := receive(s2,...);
...
send(s2,msg2,...);
...
```

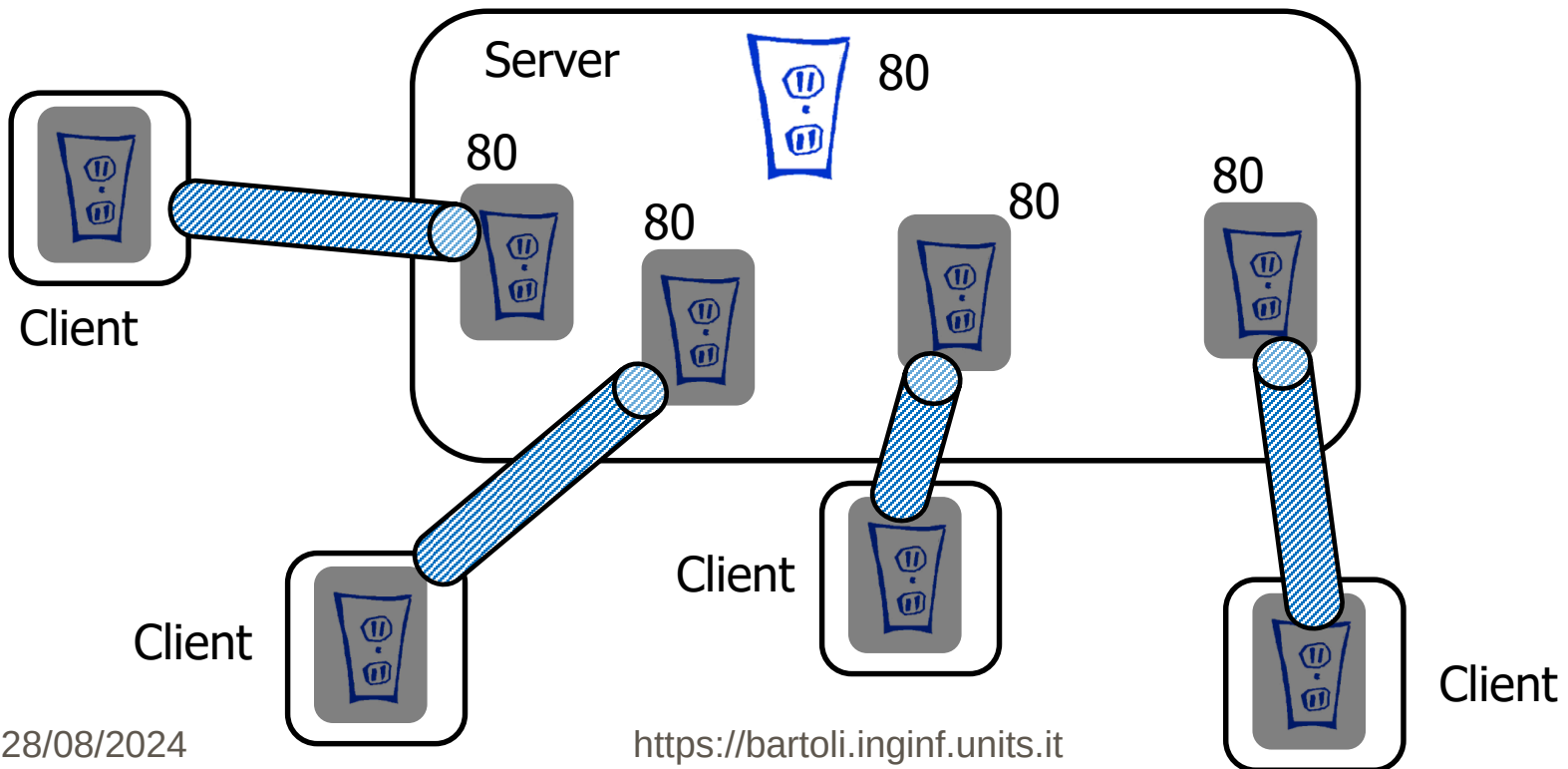
- ❑ L'applicativo:
  - ❑ **Non sceglie port number di s2**  
*(è scelto dal sistema operativo)*
  - ❑ **Non ha necessità di conoscerlo**

# Port number del Communication socket



- ❑ Communication socket e welcoming socket hanno sempre **lo stesso numero di porta**
- ❑ Il sistema operativo assegna a  $s2$  lo stesso numero di porta di  $s1$
- ❑ Come fa il sistema a non confondersi ? "Reti di Calcolatori II"

# Web server con 4 client



# Struttura di un Client (REMININD)

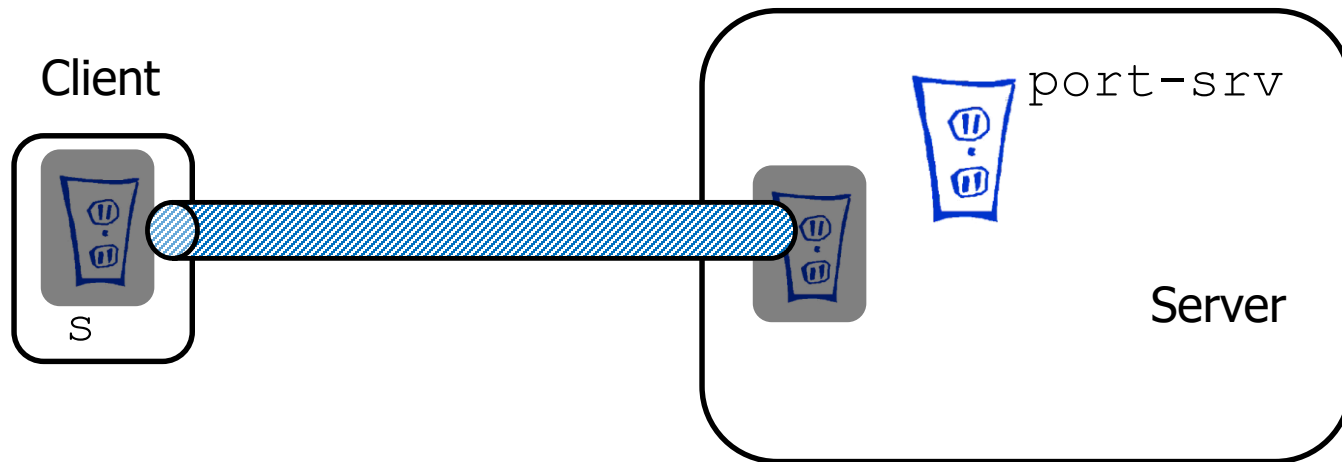
1. Determina **identificatore** del server  
(vedremo come fa ...)
2. Apre **connessione** ( $\approx$  "tubo" ) verso identificatore server
3. Esegue ciclo:
  - a) Trasmette Request
  - b) Riceve Response
4. Chiude connessione



# Client (ultra)semplificato (II)

1. Determina **identificatore** del server  
(vedremo come fa ...)
2. Apre **connessione** ( $\approx$  "tubo" ) verso identificatore server
3. Esegue ciclo send-receiveChiude connessione

- ❑ Crea socket  $s$
- ❑ Connette  $s$  con  $\langle \text{IP-srv}, \text{port-srv} \rangle$
- ❑ Comunica su  $s$  (in base ad application protocol)



# Codice (ultra)semplificato

- Crea socket  $s$
- Connette  $s$  con welcoming socket

```
int s;  
s := socket(...);  
connect(s, IP-srv, port-srv, ...);
```

```
...  
send(s, msg1, ...);  
...  
msg2 := receive(s, ...);  
...
```

Messaggi definiti nello  
application protocol

# Ultra-super-importante (Uso vs Implementazione)

```
int s;  
s := socket(...);  
connect(s, IP-srv, port-srv, ...);  
...  
send(s, msg1, ...);  
...  
msg2 := receive(s, ...);  
...
```

- ❑ L'applicativo **non** ha necessità di sapere **come sono realizzate** le primitive
- ❑ Basta sapere **come usarle**

# Client: Osservazione 1

```
int s;  
s := socket(...);  
connect(s, IP-srv, port-srv, ...);  
...  
send(s, msg1, ...);  
...  
msg2 := receive(s, ...);  
...
```

- L'applicativo non si accorge che sul lato server ci sono 2 socket  
(communication socket, welcoming socket)

# Client: Osservazione 2

```
int s;  
s := socket(...);  
connect(s, 'IP-srv, port-srv', ...);  
...  
send(s, msg1, ...);  
...  
msg2 := receive(s, ...);  
...
```

- L'applicativo:
  - **Non sceglie il proprio port number**  
*(è scelto dal sistema operativo)*
  - **Non ha necessità di conoscerlo**
- Può conoscerlo con altre funzioni, ma non gli serve

# Ricapitolando



- ❑ Server: “sceglie” il **proprio** port number
  - ❑ Client: specifica port number che cerca **sul server**
- 
- ❑ Server: non specifica da quale Client attende connessioni
  - ❑ Client: specifica <indirizzo IP, port number> del Server

# Curiosità...



- ❑ **Vedere socket utilizzati**
- ❑ Google "netstat manual"
- ❑ Windows
  - ❑ Open Command prompt
  - ❑ `netstat -a -n`

# Alcune sul mio PC...(I)

|     |                    |                     |             |
|-----|--------------------|---------------------|-------------|
| TCP | 198.18.2.218:16072 | 216.239.32.21:443   | ESTABLISHED |
| TCP | 198.18.2.218:16074 | 216.58.205.170:443  | ESTABLISHED |
| TCP | 198.18.2.218:16076 | 104.17.214.204:443  | ESTABLISHED |
| TCP | 198.18.2.218:16077 | 104.24.26.24:443    | ESTABLISHED |
| TCP | 198.18.2.218:16078 | 216.58.205.106:443  | ESTABLISHED |
| TCP | 198.18.2.218:16079 | 216.239.36.21:443   | ESTABLISHED |
| TCP | 198.18.2.218:16080 | 151.101.120.65:443  | ESTABLISHED |
| TCP | 198.18.2.218:16081 | 216.58.205.174:443  | ESTABLISHED |
| TCP | 198.18.2.218:16082 | 104.17.68.176:443   | ESTABLISHED |
| TCP | 198.18.2.218:16083 | 104.17.127.171:443  | ESTABLISHED |
| TCP | 198.18.2.218:16084 | 104.17.116.176:443  | ESTABLISHED |
| TCP | 198.18.2.218:16085 | 104.17.235.204:443  | ESTABLISHED |
| TCP | 198.18.2.218:16086 | 104.24.26.24:443    | ESTABLISHED |
| TCP | 198.18.2.218:16087 | 74.125.71.154:443   | ESTABLISHED |
| TCP | 198.18.2.218:16088 | 216.58.205.97:443   | ESTABLISHED |
| TCP | 198.18.2.218:16089 | 52.212.88.15:443    | ESTABLISHED |
| TCP | 198.18.2.218:16090 | 35.186.194.58:443   | ESTABLISHED |
| TCP | 198.18.2.218:16091 | 104.16.253.5:443    | ESTABLISHED |
| TCP | 198.18.2.218:16093 | 216.58.205.74:443   | ESTABLISHED |
| TCP | 198.18.2.218:16094 | 52.212.88.15:443    | ESTABLISHED |
| TCP | 198.18.2.218:16095 | 52.212.88.15:443    | ESTABLISHED |
| TCP | 198.18.2.218:16096 | 52.212.88.15:443    | ESTABLISHED |
| TCP | 198.18.2.218:16097 | 52.212.88.15:443    | ESTABLISHED |
| TCP | 198.18.2.218:16099 | 185.60.216.38:443   | ESTABLISHED |
| TCP | 198.18.2.218:16102 | 34.225.143.92:443   | ESTABLISHED |
| TCP | 198.18.2.218:16103 | 173.241.240.143:443 | ESTABLISHED |
| TCP | 198.18.2.218:16104 | 69.173.144.165:443  | ESTABLISHED |
| TCP | 198.18.2.218:16105 | 104.244.42.131:443  | ESTABLISHED |
| TCP | 198.18.2.218:16106 | 216.58.205.194:443  | ESTABLISHED |
| TCP | 198.18.2.218:16107 | 52.45.79.5:443      | ESTABLISHED |
| TCP | 198.18.2.218:16108 | 52.7.220.66:443     | ESTABLISHED |
| TCP | 198.18.2.218:16109 | 34.206.5.104:443    | ESTABLISHED |

# Alcune sul mio PC...(II)

|     |              |           |           |
|-----|--------------|-----------|-----------|
| TCP | 0.0.0.0:135  | 0.0.0.0:0 | LISTENING |
| TCP | 0.0.0.0:445  | 0.0.0.0:0 | LISTENING |
| TCP | 0.0.0.0:623  | 0.0.0.0:0 | LISTENING |
| TCP | 0.0.0.0:1536 | 0.0.0.0:0 | LISTENING |
| TCP | 0.0.0.0:1537 | 0.0.0.0:0 | LISTENING |
| TCP | 0.0.0.0:1538 | 0.0.0.0:0 | LISTENING |
| TCP | 0.0.0.0:1539 | 0.0.0.0:0 | LISTENING |
| TCP | 0.0.0.0:1540 | 0.0.0.0:0 | LISTENING |
| TCP | 0.0.0.0:1543 | 0.0.0.0:0 | LISTENING |

processi server  
sul mio PC?

|     |                    |                    |            |
|-----|--------------------|--------------------|------------|
| TCP | 198.18.2.218:16017 | 93.184.220.29:80   | CLOSE_WAIT |
| TCP | 198.18.2.218:16032 | 216.58.205.138:443 | CLOSE_WAIT |
| TCP | 198.18.2.218:16033 | 216.58.205.138:443 | CLOSE_WAIT |
| TCP | 198.18.2.218:16034 | 216.58.205.138:443 | CLOSE_WAIT |

cose strane...

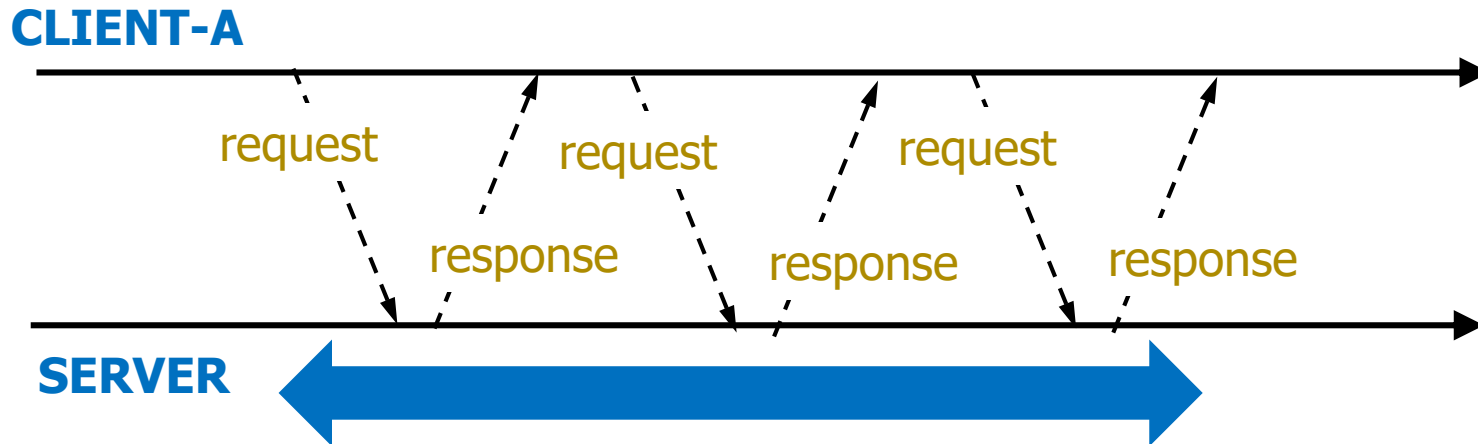
# Test - General A



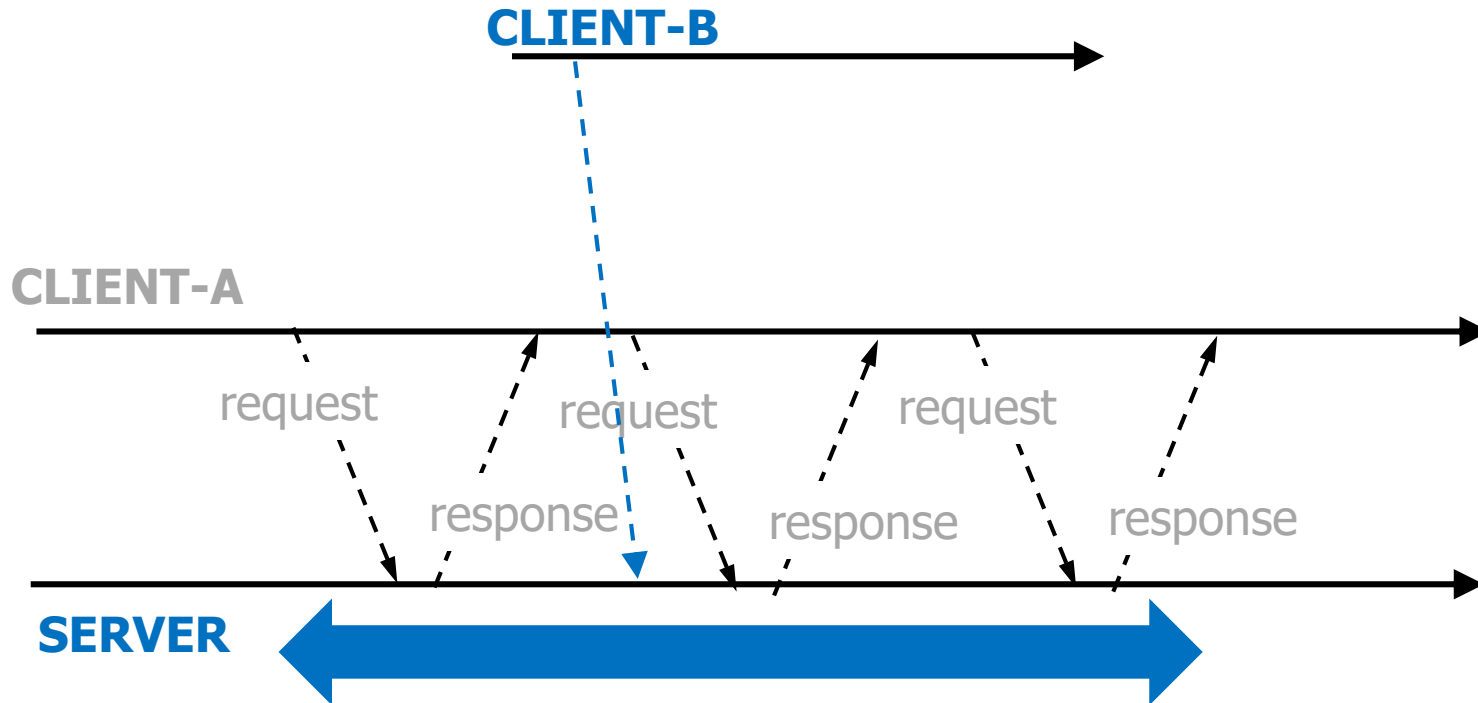
# Comprendere le scale temporali



# Caso comunissimo



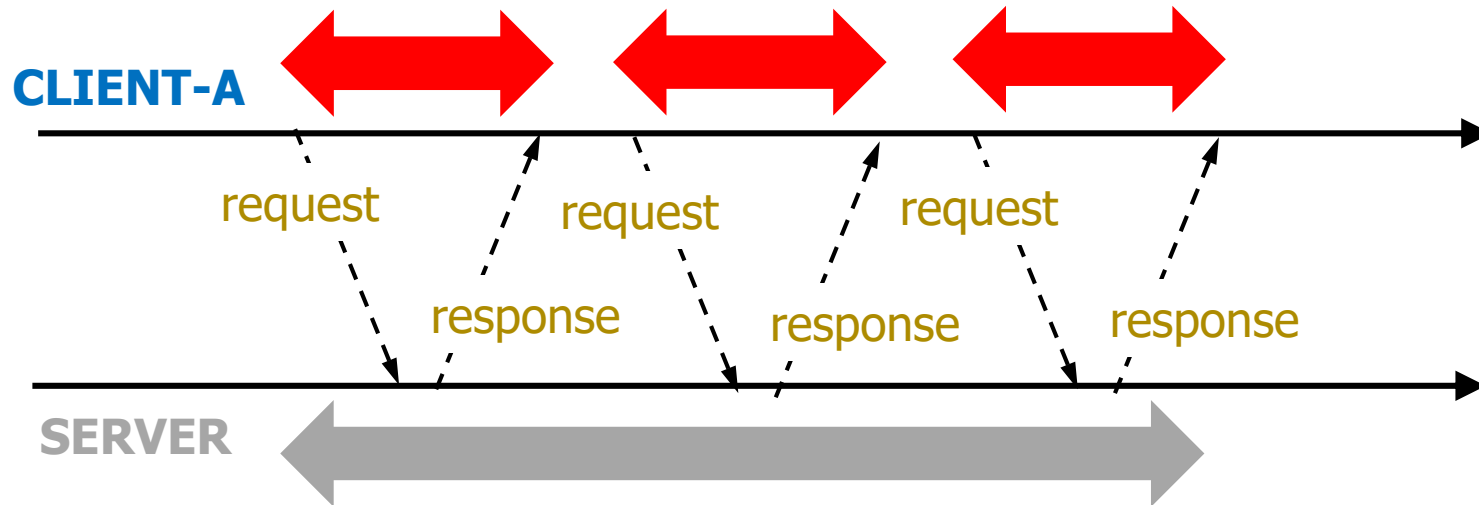
# Domanda importante 1



E' ragionevole che **Server** mantenga Client-B **in attesa** fino a quando Client-A ha chiuso la connessione?



# Domanda importante 2



E' ragionevole che **Client** rimanga **inattivo** tra invio request e ricezione response?



# Riflettiamo

ping.eu

Online Ping, Traceroute, DNS lookup, WHOIS, Port check, Reverse lookup, Proxy checker, Network mask calculator, Country by IP, Unit converter

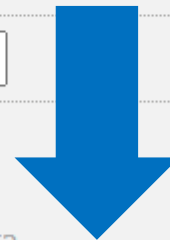
Online service Ping



**Ping** – Shows how long it takes for packets to reach host

IP address or host name:

Enter code:



--- PING www.units.it (130.186.9.3) 56(84) bytes of data. ---

64 bytes from 130.186.9.3: icmp\_seq=1 ttl=52 time=**29.9** ms

64 bytes from 130.186.9.3: icmp\_seq=2 ttl=52 time=**29.7** ms

64 bytes from 130.186.9.3: icmp\_seq=3 ttl=52 time=**29.7** ms

64 bytes from 130.186.9.3: icmp\_seq=4 ttl=52 time=**29.9** ms

# Comprendere le scale temporali (I)

| Event       | Latency |
|-------------|---------|
| 1 CPU cycle | 0.3 ns  |

|                                     |        |
|-------------------------------------|--------|
| Main memory access (DRAM, from CPU) | 120 ns |
|-------------------------------------|--------|

|                                     |       |
|-------------------------------------|-------|
| Internet: San Francisco to New York | 40 ms |
|-------------------------------------|-------|

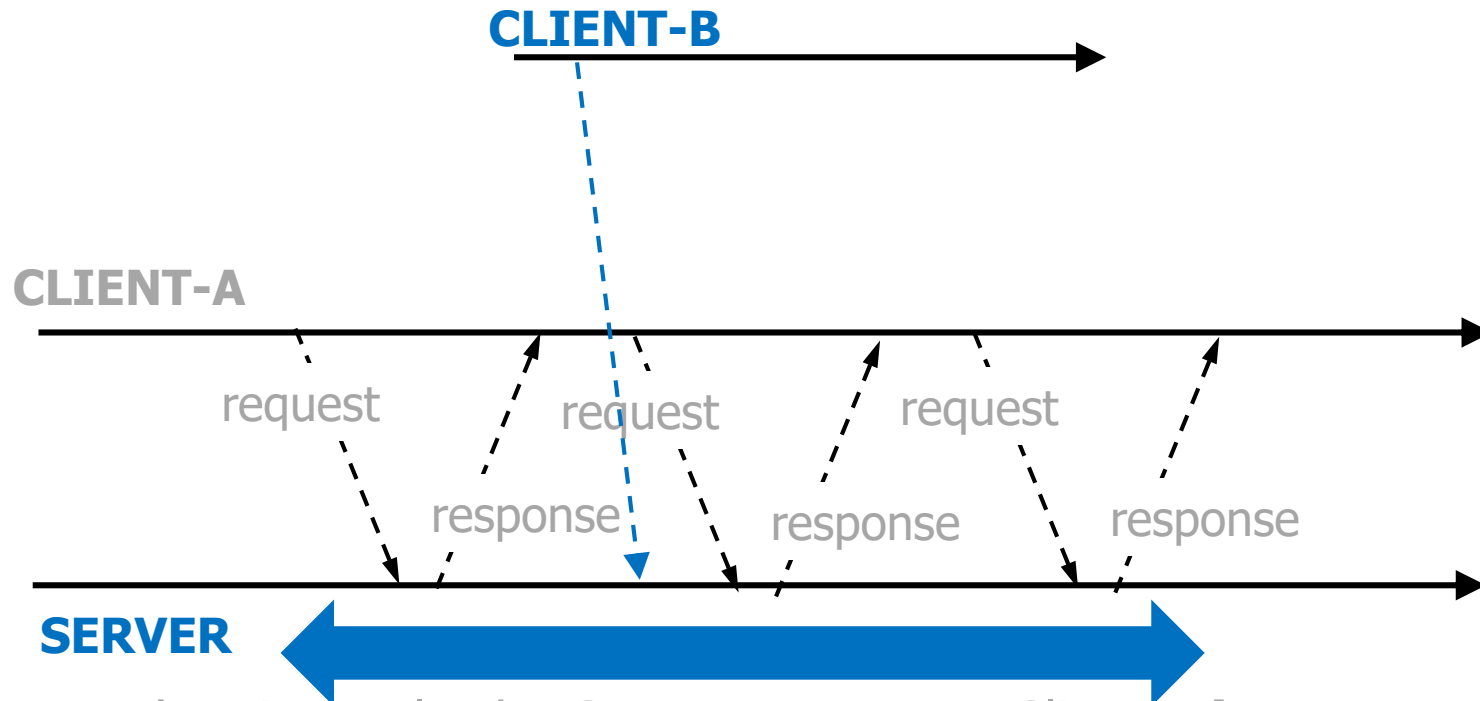
|                                           |       |
|-------------------------------------------|-------|
| Internet: San Francisco to United Kingdom | 81 ms |
|-------------------------------------------|-------|

|                                      |        |
|--------------------------------------|--------|
| Internet: San Francisco to Australia | 183 ms |
|--------------------------------------|--------|

# Comprendere le scale temporali (II)

| Event                                     | Latency | Scaled   |
|-------------------------------------------|---------|----------|
| 1 CPU cycle                               | 0.3 ns  | 1 s      |
|                                           |         |          |
| Main memory access (DRAM, from CPU)       | 120 ns  | 6 min    |
|                                           |         |          |
| Internet: San Francisco to New York       | 40 ms   | 4 years  |
| Internet: San Francisco to United Kingdom | 81 ms   | 8 years  |
| Internet: San Francisco to Australia      | 183 ms  | 19 years |
|                                           |         |          |

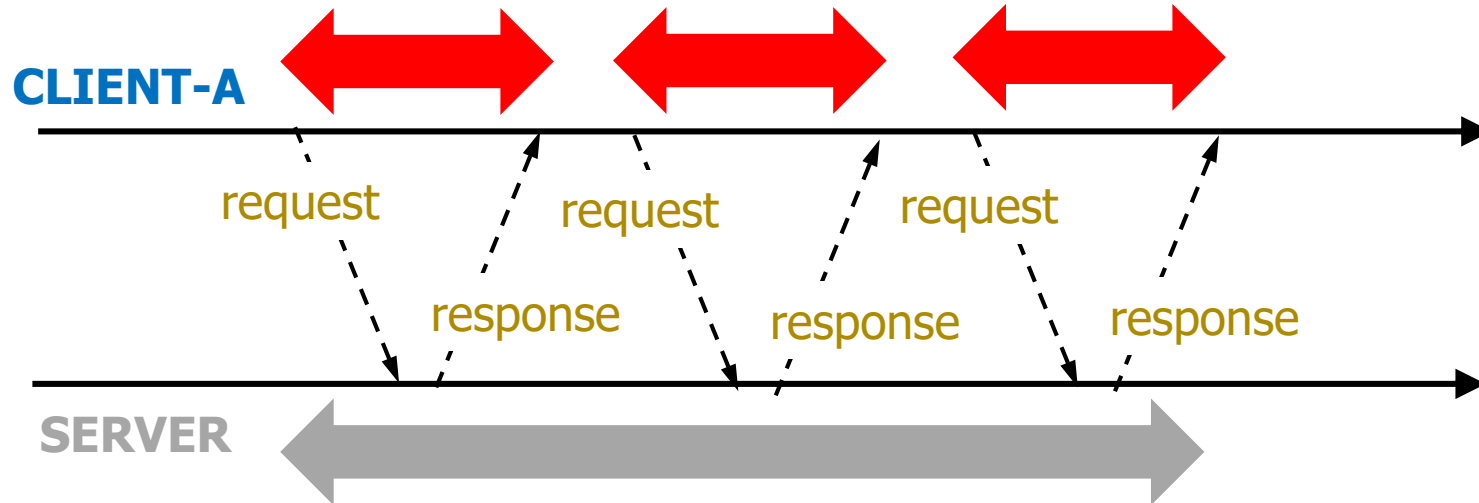
# Domanda importante 1: RISPOSTA



E' ragionevole che **Server** mantenga Client-B **in attesa** fino a quando Client-A ha chiuso la connessione? **NO!**

Server "deve" essere programmato in modo da potere gestire **più client "in parallelo"**

# Domanda importante 2: RISPOSTA



E' ragionevole che **Client** rimanga **inattivo**  
tra invio request e ricezione response? **NO!**

Client "deve" essere programmato in modo da  
potere effettuare **più attività "in parallelo"**  
(se ci sono altre attività da fare)

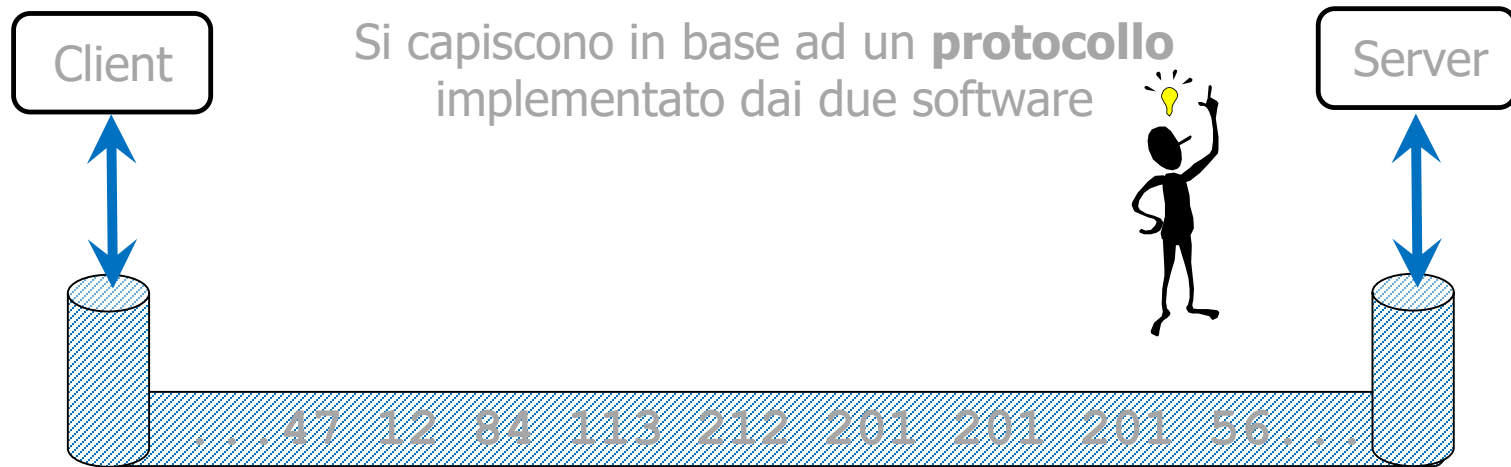
# Curiosità: Dischi SSD

| Event                               | Latency        | Scaled      |
|-------------------------------------|----------------|-------------|
| 1 CPU cycle                         | 0.3 ns         | 1 s         |
| Solid-state disk I/O (flash memory) | 50–150 $\mu$ s | 2–6 days    |
| Rotational disk I/O                 | 1–10 ms        | 1–12 months |

# Protocolli di testo vs binari



# Protocolli (REMIND)



- Due categorie
  - Testo
  - Binari
- Vediamo che significa
- Riprendiamo un pò di fondamenti di informatica...

# Numeri vs Rappresentazioni

|                                          |                                        |         |
|------------------------------------------|----------------------------------------|---------|
| ❑ Numeri                                 | Esistono nella nostra mente            |         |
| ❑ Rappresentazione dei numeri            | Disegnini (ci fanno pensare ai numeri) |         |
| ❑ Rappresentazione comune                | base 10                                | 94      |
| ❑ Cifre: 0 1 2 3 4 5 6 7 8 9             |                                        |         |
| ❑ Rappresentazione meno comune           | base 2                                 | 1001001 |
| ❑ Cifre: 0 1 (" <b>bit</b> ")            |                                        |         |
| ❑ Rappresentazione meno comune           | base 16                                | 3AB2    |
| ❑ Cifre: 0 1 2 3 4 5 6 7 8 9 A B C D E F |                                        |         |

# Rappresentazioni (I)

## ❑ Algoritmi (noiosi) per **convertire** tra rappresentazioni

- ❑ Disegno che rappresenta N in base 10 →  
Disegno che rappresenta **lo stesso** N in base 2
- ❑ Disegno che rappresenta N in base 2 →  
Disegno che rappresenta **lo stesso** N in base 10

## ❑ Base 2 → Base 10 si fa "a colpo d'occhio"

- ❑ Fate pratica! Basta imparare le potenze di 2

- |         |          |
|---------|----------|
| ❑ 1001  | 1+8      |
| ❑ 1100  | 4+8      |
| ❑ 11101 | 1+4+8+16 |

# Rappresentazioni (II)

- ❑ Proprietà (noiose) per determinare **quante cifre** occorrono
  - ❑ Quanti numeri rappresento con 4 cifre in base 10?
    - ❑ Intuizione: Da 0 a 9999
    - ❑ Formula:  $[0, 10^4 - 1]$
  - ❑ Quanti numeri rappresento con 6 cifre in base 2?
    - ❑ Facile da ricordare:  $[0, 2^6 - 1]$
  - ❑ Quanti numeri rappresento con 7 cifre in base 16?
    - ❑ Facile da ricordare:  $[0, 16^7 - 1]$

# Byte



- ❑ 8 cifre in base 2
- ❑ 2 cifre in base 16
  
- ❑ Quali naturali rappresento con 8 cifre in base 2?  
 $[0, 2^8-1] = [0, 255]$
  
- ❑ Quali naturali rappresento con 2 cifre in base 16?  
 $[0, 16^2-1] = [0, 255]$

# Conversione "a colpo d'occhio"

□ **Byte in base 2 ↔ Byte in base 16**

□ Procedimento (dimostrazione omessa)

1. 4 bit / 1 cifra base 16
2. Passaggio da base 10 a colpo d'occhio
3. Affianco i risultati

□ A2                    10100010

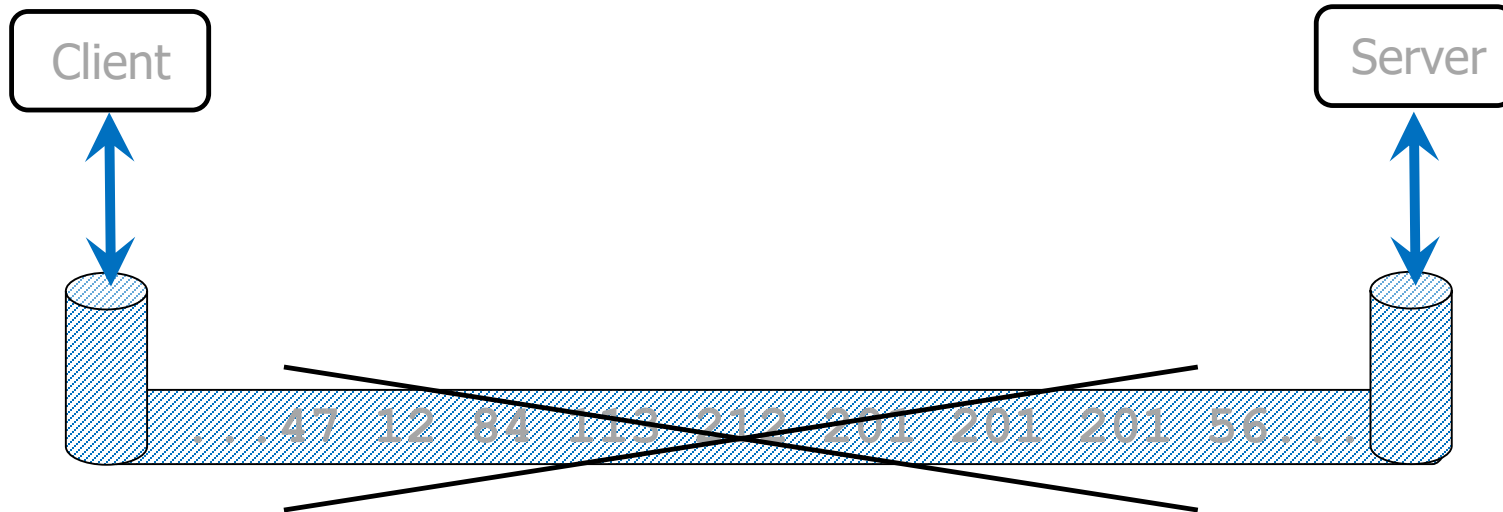
□ 3B                    00111011

□ 34                    00110100

□ 10010011          93

□ 10001010          8A

# Perché vi dico queste cose noiosissime?



- ❑ Una connessione trasporta solo **sequenze di byte**
- ~~❑ 1 byte (8 bit) rappresenta un **numero naturale**  $[0, 255]$~~
- ❑ **Traffico** rappresentato "**sempre**" come **byte in base 16**

# Esempio

> Frame 9: 187 bytes on wire (1496 bits)

```
00 0c 29 61 d8 3e 00 0c 29 70 c6 23 08 00 45 00
00 ad ed 64 40 00 40 06 d3 01 ac 10 10 dd ac 10
10 e7 dd 76 00 19 81 88 6d 79 d6 c3 ef d6 80 18
00 ed 41 7e 00 00 01 01 08 0a 01 53 80 0e 01 53
43 7c 4d 41 49 4c 20 46 52 4f 4d 3a 3c 73 61 6e
64 65 72 73 40 73 6b 79 6e 65 74 2e 6c 6f 63 61
6c 3e 20 53 49 5a 45 3d 37 33 32 0d 0a 52 43 50
54 20 54 4f 3a 3c 73 61 6e 64 65 72 73 40 63 79
62 65 72 64 79 6e 65 2e 6c 6f 63 61 6c 3e 20 4f
52 43 50 54 3d 72 66 63 38 32 32 3b 73 61 6e 64
65 72 73 40 63 79 62 65 72 64 79 6e 65 2e 6c 6f
63 61 6c 0d 0a 44 41 54 41 0d 0a
```

Byte che  
compongono il  
messaggio

"Facile" capire  
quali bit sono 0 e  
quali sono 1  
(se serve)

# Testo vs Codifica (I)

## ❑ Testo

## ❑ Codifica del testo in un calcolatore

"Disegnini"

Numeri naturali

## ❑ Codifica comune: ASCII

❑ 1 byte  $\leftrightarrow$  1 "disegnino"

❑ 256 "disegnini" codificabili

|   |    |   |    |
|---|----|---|----|
| - | 2D | L | 4C |
| . | 2E | M | 4D |
| / | 2F | N | 4E |
| 0 | 30 | O | 4F |
| 1 | 31 | P | 50 |
| 2 | 32 | Q | 51 |
| 3 | 33 |   |    |

❑ Tastiera: premo disegno

❑ Schermo: **"esce"** numero naturale

→ **"entra"** numero naturale

→ visualizzato disegno

# Testo vs Codifica (II)

□ **Linea** di testo: sequenza di caratteri **terminata** da:

- "fine linea"
- "vai a capo"
- "fine linea" "vai a capo"
- "vai a capo" "fine linea"

Vari casi possibili, purtroppo (non approfondiamo)



```
marco van basten  
ruud gullit
```

```
6D 61 72 63 6F 20 76 61 6E  
20 62 61 73 74 65 6E 0A 72  
75 75 64 20 67 75 6C 6C 69  
74
```

# Protocolli di testo vs binari



- ❑ Protocolli di testo
  - ❑ Messaggi sono **linee di testo**
  - ❑ Esempi: HTTP (web), SMTP POP (email)
  
- ❑ Protocolli binari
  - ❑ Messaggi **non** sono linee di testo
  - ❑ Ogni bit ha un significato specifico
  
  - ❑ Esempi: DNS

# Esempio

## HTTP, testo

Byte che compongono il  
messaggio

Codifica ASCII

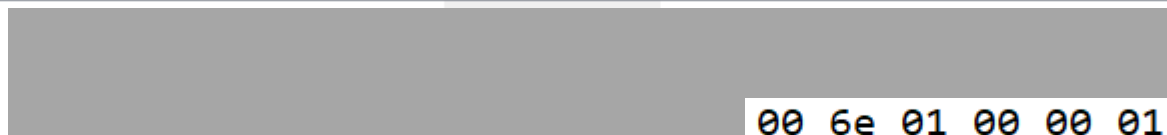
| Byte che compongono il messaggio |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    | Codifica ASCII |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |  |
|----------------------------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----------------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|--|
| 6f                               | 6e | 2e | 69 | 63 | 6f | 20 | 48 | 54 | 54 | 50 | 2f | 31 | 2e | 31 | 0d | GE             | T | / | f | a | v | i | c |   |   |   |   |   |   |   |   |   |  |
| 0a                               | 48 | 6f | 73 | 74 | 3a | 20 | 67 | 54 | 54 | 50 | 2f | 31 | 2e | 31 | 0d | o              | n | . | i | c | o |   | H |   | T | T | P | / | 1 | . | 1 |   |  |
| 6d                               | 61 | 73 | 73 | 2e | 65 | 64 | 75 | 61 | 69 | 61 | 2e | 63 | 73 | 2e | 75 | ·              | H | o | s | t | : |   | g |   | a | i | a | . | c | s | . | u |  |
| 67                               | 65 | 6e | 74 | 3a | 20 | 4d | 6f | 0d | 0a | 55 | 73 | 65 | 72 | 2d | 41 | mass           | . | e | d | u |   | · | U | s | e | r | - | A |   |   |   |   |  |
| 30                               | 20 | 28 | 57 | 69 | 6e | 64 | 6f | 7a | 69 | 6c | 6c | 61 | 2f | 35 | 2e | gent           | : |   | M | o |   | z | i | l | l | a | / | 5 | . |   |   |   |  |
| 69                               | 6e | 64 | 6f | 77 | 73 | 20 | 4e | 77 | 73 | 3b | 20 | 55 | 3b | 20 | 57 | 0              | ( | W | i | n | d | o | w | s | ; | U | ; | W |   |   |   |   |  |
| 6e                               | 2d | 55 | 53 | 3b | 20 | 72 | 76 | 54 | 20 | 35 | 2e | 31 | 3b | 20 | 65 | i              | n | d | o | w | s |   | N | T |   | 5 | . | 1 | ; | e |   |   |  |
| 47                               | 65 | 63 | 6b | 6f | 2f | 32 | 30 | 3a | 31 | 2e | 30 | 2e | 32 | 29 | 20 | n              | - | U | S | ; |   | r | v |   | : | 1 | . | 0 | . | 2 | ) |   |  |
| 65                               | 74 | 73 | 63 | 61 | 70 | 65 | 2f | 30 | 32 | 31 | 31 | 32 | 30 | 20 | 4e | G              | e | c | k | o | / | 2 | 0 |   | 0 | 2 | 1 | 1 | 2 | 0 |   | N |  |
| 63                               | 65 | 70 | 74 | 3a | 20 | 74 | 65 | 37 | 2e | 30 | 31 | 0d | 0a | 41 | 63 | e              | t | s | c | a | p | e | / | 7 | . | 0 | 1 | · | A | c |   |   |  |
| 70                               | 70 | 6c | 69 | 63 | 61 | 74 | 69 | 78 | 74 | 2f | 78 | 6d | 6c | 2c | 61 | c              | e | p | t | : |   | t | e | x | t | / | x | m | l | , | a |   |  |
| 70                               | 70 | 6c | 69 | 63 | 61 | 74 | 69 | 6f | 6e | 2f | 78 | 6d | 6c | 2c | 61 | p              | p | l | i | c | a | t | i | o | n | / | x | m | l | , | a |   |  |
| 2b                               | 78 | 6d | 6c | 2c | 74 | 65 | 78 | 74 | 2f | 68 | 74 | 6d | 6c | 3b | 71 | p              | p | l | i | c | a | t | i | o | n | / | x | h | t | m | l |   |  |
| 3d                               | 30 | 2e | 39 | 2c | 74 | 65 | 78 | 74 | 2f | 70 | 6c | 61 | 69 | 6e | 3b | +x             | m | l | , | t | e | x | t | / | h | t | m | l | ; | q |   |   |  |
| 71                               | 3d | 30 | 2e | 38 | 2c | 76 | 69 | 64 | 65 | 6f | 2f | 78 | 2d | 6d | 6e | =              | 0 | . | 9 | , | t | e | x | t | / | p | l | a | i | n | ; |   |  |
| 67                               | 2c | 69 | 6d | 61 | 67 | 65 | 2f | 70 | 6e | 67 | 2c | 69 | 6d | 61 | 67 | q              | = | 0 | . | 8 | , | v | i | d | e | o | / | x | - | m | n |   |  |
| 65                               | 2f | 6a | 70 | 65 | 67 | 2c | 69 | 6d | 61 | 67 | 65 | 2f | 67 | 69 | 66 | g              | , | i | m | a | g | e | / | p | n | g | , | i | m | a | g |   |  |
|                                  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    | e              | / | j | p | e | g | , | i | m | a | g | e | / | g | i | f |   |  |

*Fine linea*

# Esempio

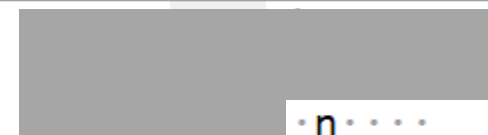
## DNS, binario (I)

Byte che compongono il  
messaggio



00 00 00 00 00 00 03 77 77 77 00 6e 01 00 00 01  
6f 72 67 00 00 01 00 01 04 69 65 74 66 03

Codifica ASCII



.....n.....  
.....w ww.ietf.  
org.....

- ❑ Alcuni byte hanno valori che corrispondono a caratteri ASCII stampabili
- ❑ ...ma il significato del messaggio **non** si ottiene interpretandolo come testo

# Esempio

## DNS, binario (II)

Byte che compongono il  
messaggio

```
00 00 00 00 00 00 03 77 77 77 04 69 65 74 66 03
6f 72 67 00 00 01 00 01
```

Codifica ASCII

```
.....n.....
.....w ww.ietf.
org.....
```

```
Domain Name System (query)
Transaction ID: 0x006e
Flags: 0x0100 Standard query
  0... .. = Response: Message is a query
  .000 0... .. = Opcode: Standard query (0)
  .... ..0. .... = Truncated: Message is not truncated
  .... ..1 .... = Recursion desired: Do query recursively
  .... ..0.. .... = Z: reserved (0)
  .... ..0 .... = Non-authenticated data: Unacceptable
Questions: 1
Answer RRs: 0
Authority RRs: 0
Additional RRs: 0
Queries
[Response In: 9]
```

Ogni bit ha un  
significato specifico

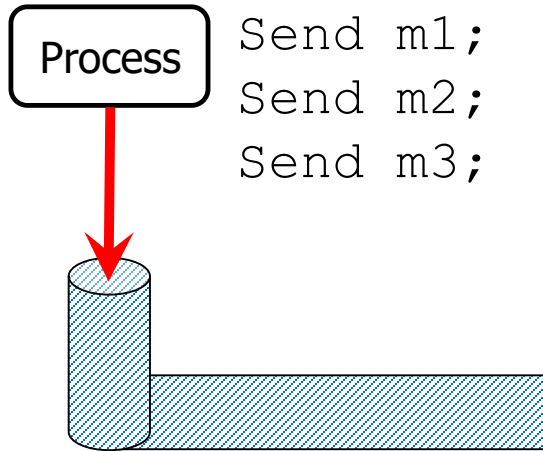
# Google Colab Notebook



# **Proprietà Servizi Comunicazione**

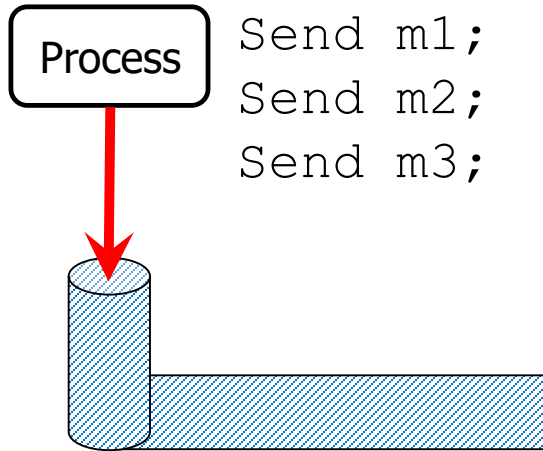


# Proprietà Servizi Comunicazione (I-a)



- ❑ Arriveranno **tutti**?
- ❑ Potrebbero arrivare in **ordine diverso** da come li ho trasmessi?
- ❑ Potrebbero arrivare **più** volte?
- ❑ In quanto **tempo**?

# Proprietà Servizi Comunicazione (I-b)



Servizio di comunicazione:

- ☐ Quali garanzie offre a chi lo utilizza?
- ☐ **Non** interessa realizzazione (funzionamento)
- ☐ Interessa solo come appare dall'esterno (**uso**)

# Proprietà Servizi Comunicazione (II)



Un servizio di comunicazione può apparire:

- ❑ **Connection-oriented** oppure **Connectionless**
- ❑ **Byte-oriented** oppure **Message-oriented**
- ❑ **Reliable** oppure **Unreliable**

# Connection-oriented vs...



1. **Apertura** della connessione (“creazione tubo virtuale”)
  2. Trasmetto / ricevo **nella connessione**
  3. **Chiusura** della connessione
- 
- ☐ Necessaria apertura esplicita **prima** dell’utilizzo
  - ☐ Specifico destinatario **solo** alla apertura
  - ☐ **Mantiene ordinamento**
    - ☐ Ordine di ricezione = ordine di trasmissione

# ...vs Connectionless

- 
1. **Apertura** della connessione ("creazione tubo virtuale")
  2. Trasmetto / ricevo **nella connessione**
  3. **Chiusura** della connessione

- ☐ Non occorre apertura preventiva di nulla
- ☐ Ogni trasmissione indica il destinatario
- ☐ Ogni ricezione indica il trasmittente
- ☐ Può non mantenere ordinamento

# Message vs. byte



## Byte-oriented

- ❑ Trasmissioni consecutive creano un unico flusso di byte (la nozione di "messaggio" non esiste)

## Message-oriented

- ❑ Ogni trasmissione è un messaggio separato dagli altri

Sender esegue 2 send da 1KB ognuna

- ❑ Byte-oriented: Receiver vede **1 sequenza di byte** lunga 2KB
- ❑ Message-oriented: Receiver vede **2 messaggi** da 1KB ognuno

# Reliability (I)



## Reliable

- ☐ Tutti i dati trasmessi sono ricevuti  
AND
- ☐ Ogni dato trasmesso viene ricevuto **solo una volta**

## Unreliable

- ☐ Alcuni dati trasmessi possono essere **persi**  
OR
- ☐ Alcuni dati potrebbero essere ricevuti **più volte (duplicati)**

# Reliability (II)



## IMPORTANTISSIMO

*"Tutti i dati trasmessi vengono ricevuti"*     **irrealistico !**

- ❑ Un servizio reliable fa del suo meglio per non perdere messaggi
- ❑ Se li perde, o **potrebbe** averli persi, avvisa con "connessione caduta"

Esempio (reliable connection):

- ❑ Il servizio può decidere di rompere la connessione
- ❑ Finché non la rompe, fa arrivare tutti i messaggi e li fa arrivare in ordine
- ❑ Il servizio la rompe "quando non ne può fare a meno"

# Combinazioni tipiche

Applicativo



TCP

Proprietà viste dall'applicativo

- ☐ Connection
- ☐ Reliable
- ☐ Byte-oriented

Applicativo



UDP

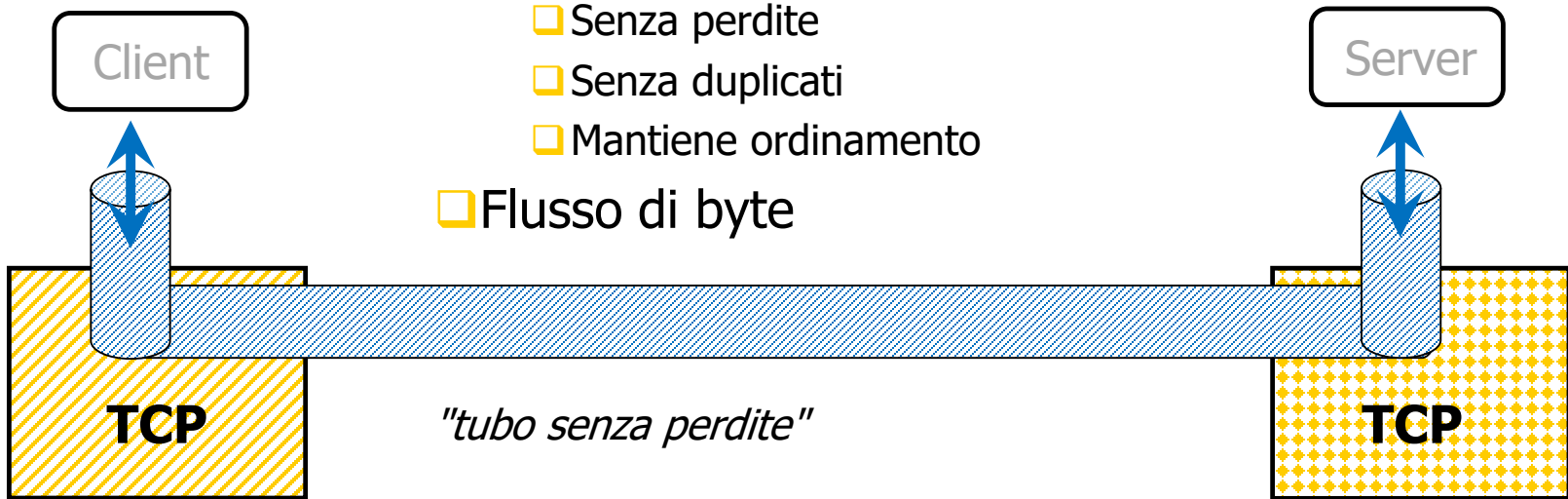
Proprietà viste dall'applicativo

- ☐ Connection**less**
- ☐ **Un**reliable
- ☐ **Message**-oriented

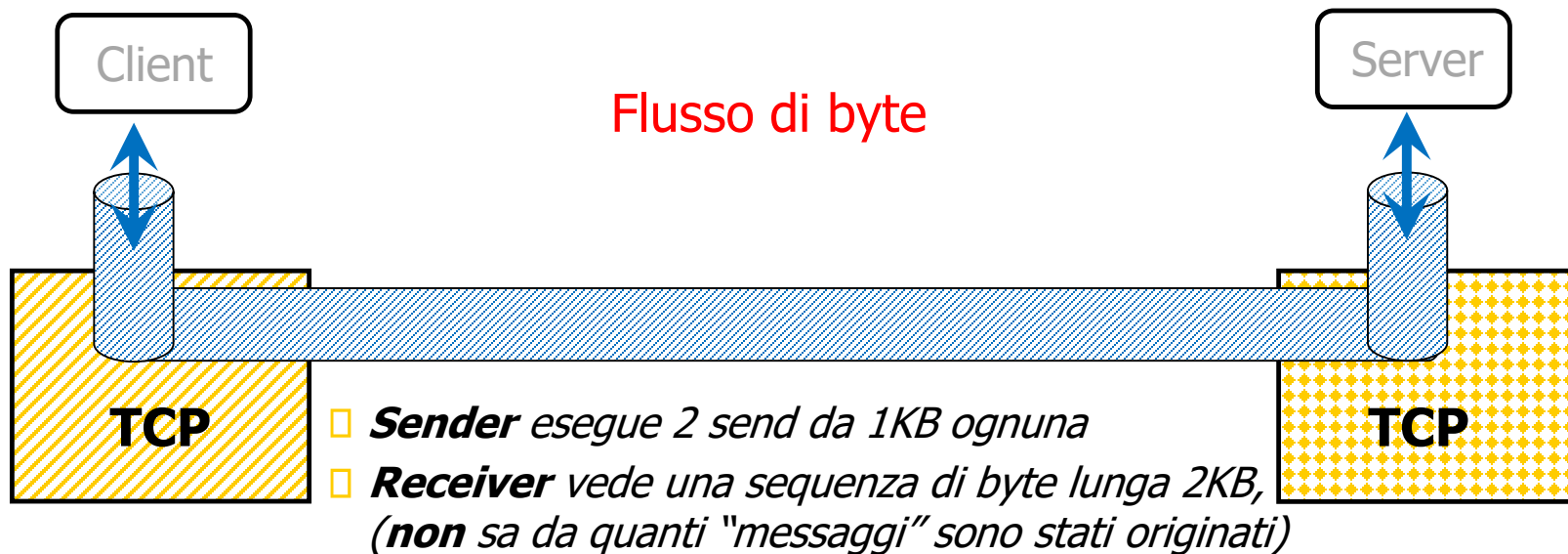
Applicativo deve essere preparato a gestire questi eventi

# Servizio TCP

- ❑ Connessione
- ❑ Comportamento "ideale"
  - ❑ Senza perdite
  - ❑ Senza duplicati
  - ❑ Mantiene ordinamento
- ❑ Flusso di byte

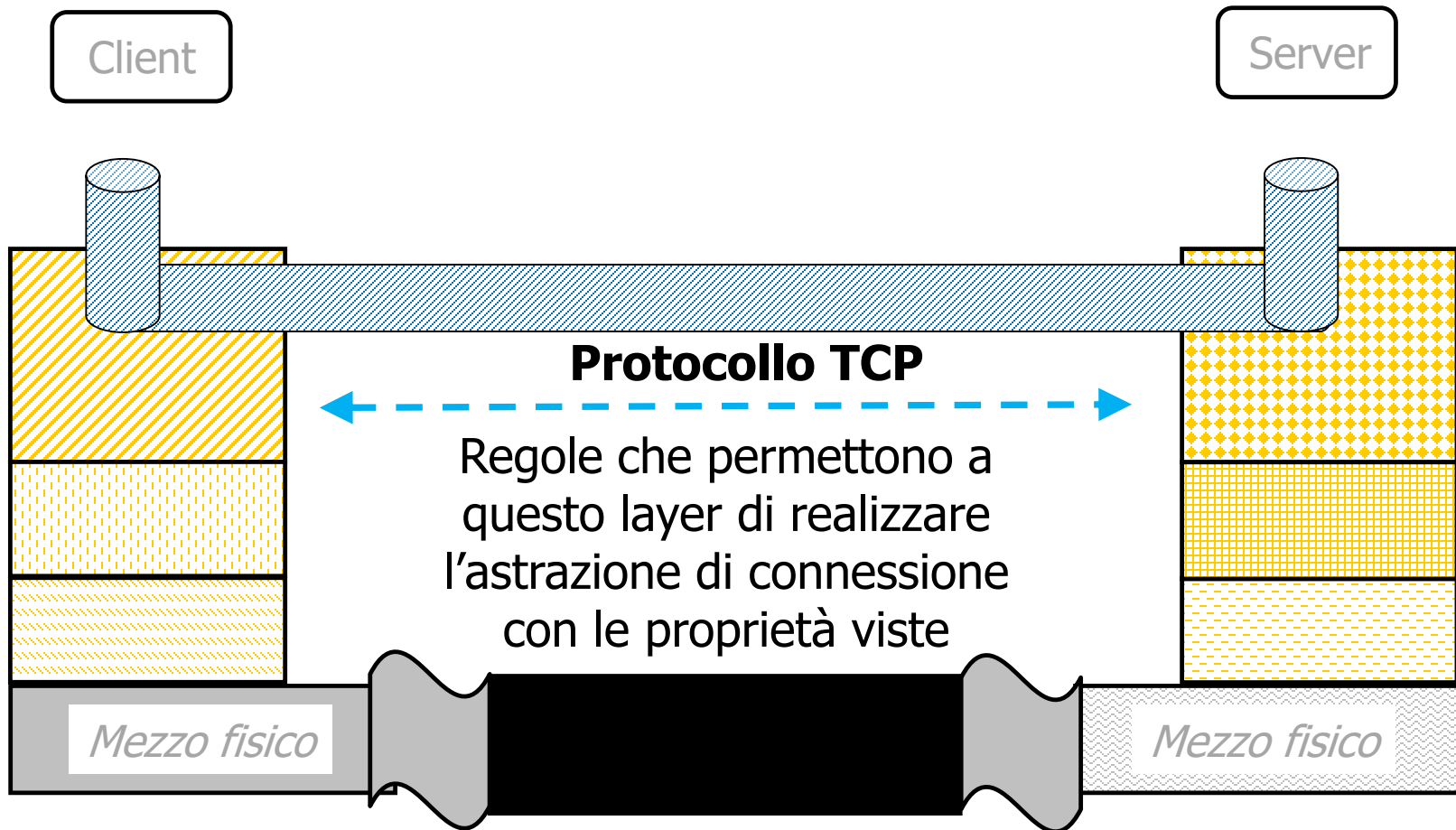


# Servizio TCP: Osservazione



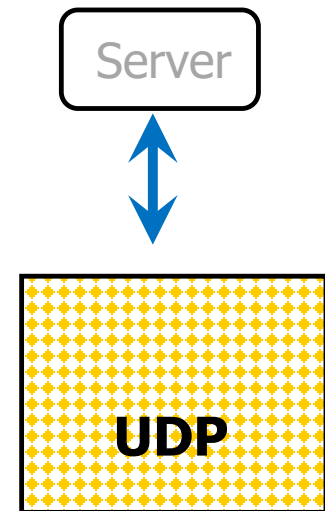
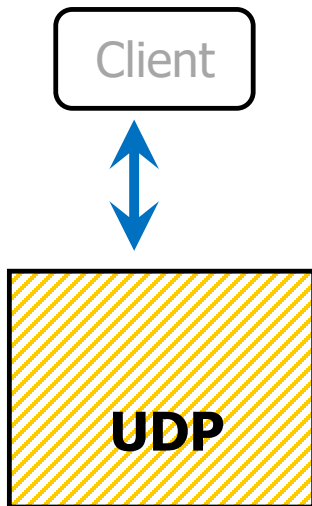
**Ogni** protocollo applicativo che usa TCP  
**deve** avere regole per "affettare" il  
flusso di byte in messaggi

# Protocollo TCP



# Servizio UDP

- Non occorre creare previamente nulla
  - Ogni dato identifica mittente e destinatario
- Può succedere di tutto
  - Perdite
  - Duplicati (cloni indistinguibili)
  - Alterazione ordinamento
- Flusso di messaggi



# Importantissimo



- ❑ Un servizio di comunicazione può apparire:
- ❑ **Connection-oriented** oppure **Connectionless**
- ❑ **Byte-oriented** oppure **Message-oriented**
- ❑ **Reliable** oppure **Unreliable**
- ❑ **Nessuna** proprietà relativa al **tempo**
- ❑ I servizi di comunicazione usati in Internet non danno nessuna garanzia di ordine temporale

# Test General - B

